



SAPIENZA
UNIVERSITÀ DI ROMA

iBEARer: Una Libreria Flessibile per Manipolazioni Sperimentali sui Social Media

Facoltà di Ingegneria dell'Informazione, Informatica e Statistica
Laurea in Informatica

Francesco Giammaria
Matricola 2108040

A handwritten signature in black ink, appearing to read "Francesco Giammaria".

Relatore
Prof. Mattia Samory

A handwritten signature in black ink, appearing to be a stylized "M" followed by a flourish.

Anno Accademico 2025/2026

Tesi non ancora discussa

iBEARer: Una Libreria Flessibile per Manipolazioni Sperimentali sui Social Media

Tesi di Laurea Triennale. Sapienza Università di Roma

© 2026 Francesco Giammaria. Tutti i diritti riservati

Questa tesi è stata composta con L^AT_EX e la classe Sapthesis.

Email dell'autore: giammaria.2108040@studenti.uniroma1.it

Abstract

Ad oggi, gli algoritmi delle piattaforme social ordinano i contenuti per massimizzare l'*engagement* e, così facendo, favoriscono la polarizzazione e la disinformazione. Lo studio di questi fenomeni richiede esperimenti in tempo reale e, soprattutto, nell'ambiente nativo della piattaforma, per non compromettere le abitudini comportamentali degli utenti e preservare la validità ecologica. Tuttavia, la progressiva chiusura delle API proprietarie ed i moderni standard di sicurezza dei browser, impediscono l'intercezione del traffico di rete, ostacolando la ricerca ed invalidando alcuni approcci esistenti. Tali approcci risultano inoltre legati a singole piattaforme, con un elevato accoppiamento tra logica sperimentale ed architetturale, che ne compromettono la riproducibilità.

Questa tesi presenta iBEARer (*in-Browser Experiments on Algorithmic Re-Ranking*), un framework progettato per la conduzione di *Algorithmic Audit Studies* indipendenti sui social media. L'architettura elimina la necessità di accordi di collaborazione con le piattaforme, i quali spesso vincolano la pubblicazione dei risultati a veti contrattuali. Inoltre, il sistema disaccoppia la logica sperimentale dalla logica della piattaforma, favorendo la riproducibilità e la condivisione degli esperimenti.

Il framework permette di operare nel rispetto dei limiti di sicurezza per le chiamate di rete, applicando manipolazioni direttamente lato client. Infine, permette di gestire l'intero ciclo di vita dell'esperimento, dalla somministrazione dei questionari, all'esecuzione degli interventi, fino alla raccolta della telemetria su un server esterno.

Per validare l'approccio proposto si realizza poi uno studio pilota su Reddit, con lo scopo di misurare l'impatto che il *framing* di un contenuto ha sull'*engagement* degli utenti. In questo modo, i risultati dimostrano la capacità del sistema di supportare una ricerca comportamentale indipendente, riproducibile e invisibile all'utente.

Indice

1	Introduzione	1
1.1	Motivazione	1
1.2	Il Divario Tecnologico	2
1.3	Dichiarazione del Problema	3
1.4	Novità dell'Approccio	3
1.5	Contributo	3
1.6	Implicazioni	4
2	Related Work	5
2.1	Algorithmic Auditing	5
2.2	Approcci Esistenti: Stato dell'Arte (SOTA)	6
2.3	Limiti degli Approcci Esistenti	6
3	Formalizzazione e Definizioni	9
3.1	User Journey	9
3.2	Requisiti del Framework	10
3.3	Definizioni	11
3.3.1	Esperimenti Comportamentali Controllati ed A/B Test . . .	11
3.3.2	Formalizzazione dell'Esperimento	12
3.3.3	Formalizzazione di Telemetria ed Eventi	12
4	Metodologia	13
4.1	Architettura Software	13
4.2	Configurazione dell'Esperimento	14
4.2.1	Disaccoppiamento del Design Sperimentale	14
4.2.2	Disaccoppiamento dalla Piattaforma Ospite	15
4.3	Comunicazione e Flussi Dati	15
4.3.1	Comunicazioni Client-Server	15
4.3.2	Gestione della Privacy dei Partecipanti	16
4.3.3	Gestione del Ciclo di Vita dei Partecipanti	17
4.4	Esempio Pratico di Applicazione	19
4.4.1	Installazione ed Enrollment	19
4.4.2	Configurazione dell'Esperimento	19
4.4.3	Raccolta della Telemetria	19
4.4.4	Fine dell'Esperimento	19
4.5	Sfide Tecniche	20

4.6	Soluzioni Architettureali	20
4.6.1	Robustezza degli Interventi sul DOM	20
4.6.2	Gestione dell'Infinite Scroll	21
4.6.3	Gestione delle SPA	22
4.6.4	Isolamento dell'Ambiente Globale	22
4.6.5	Iniezione Ibrida dei Contenuti	22
4.7	Proprietà Architettureali ed Estensibilità	24
4.7.1	Estensione ad un Nuovo Esperimento sulla Stessa Piattaforma	24
4.7.2	Aggiunta di Nuovi Interventi sulla Stessa Piattaforma	24
4.7.3	Estensione dell'Esperimento su una Nuova Piattaforma	24
5	Studio Empirico	27
5.1	Design Sperimentale	27
5.1.1	Ipotesi di Ricerca	27
5.1.2	Configurazione dello Studio	28
5.1.3	Variabili e Moderatori	28
5.2	Metodo di Analisi	28
5.3	Risultati	29
5.3.1	HP1: Nessun Effetto del Framing	29
5.3.2	HP2-HP4: Nessuna Moderazione	30
5.3.3	Effetti Diretti dei Trattati sull'Engagement	31
5.3.4	Analisi Post-Hoc: Sottoinsiemi per Tipo di Contenuto	32
5.4	Sintesi	32
6	Discussione	33
6.1	Manipolazione Post-Rendering	33
6.2	Determinismo e Latenza	34
6.3	Auditing Indipendente	34
6.4	Riproducibilità ed Estendibilità	35
6.4.1	Implicazioni per la Ricerca	35
7	Limiti e Sviluppi Futuri	37
7.1	Fragilità dell'Euristica DOM	37
7.1.1	Sviluppo Futuro: Riconoscimento tramite Computer Vision	37
7.2	Latenza di Rete nell'Iniezione Dinamica	37
7.2.1	Sviluppo Futuro: Esecuzione nel Browser	38
7.3	Desincronizzazione dei Contenuti	38
7.3.1	Sviluppo Futuro: Forzare la Sincronizzazione	38
7.4	Ambiente di Utilizzo	39
7.4.1	Sviluppo Futuro: Estensione ad Ambienti Mobile	39
7.5	Validazione Single-Platform	39
7.5.1	Sviluppo Futuro: Estensione a Nuove Piattaforme	39
8	Conclusioni	41
	Bibliografia	43

Capitolo 1

Introduzione

1.1 Motivazione

L'ordinamento algoritmico (*algorithmic ranking*) è una delle tecnologie principali che guida l'accesso all'informazione in tutti i sistemi digitali [5]. Questa tecnologia è onnipresente: ordina i risultati nei motori di ricerca, ordina i prodotti sulle piattaforme di e-commerce, seleziona i brani nei servizi di streaming musicale, personalizza gli annunci nella pubblicità, e così e via... Nel contesto dei social media, le piattaforme utilizzano l'*algorithmic ranking* per selezionare ed ordinare i contenuti presenti nel *feed* dell'utente [1].

Attraverso la selezione e l'ordinamento dei post, gli algoritmi di ranking influenzano le opinioni, le emozioni e i comportamenti degli utenti [5] rappresentando a tutti gli effetti fonti di informazione primarie per un'ampia frazione della popolazione globale [5], prevalentemente giovani (Generazione Z e Millennials) per temi di attualità, politica, intrattenimento, e recensioni di brand o prodotti.

Tuttavia, gli algoritmi commerciali utilizzati dalle Piattaforme puntano sempre di più a massimizzare l'*engagement* (il livello di interazione e il tempo di permanenza dell'utente) trascurando completamente le ricadute sociali. La letteratura accademica conferma che questo tipo di approccio favorisce la polarizzazione affettiva e facilita l'adesione a ideologie radicali o gruppi problematici, come teorie del complotto o sette [6]. L'abbandono di tali comunità online richiede un processo di disimpegno costante. Tale processo risulta facilitato dall'esposizione a evidenze scientifiche o fattuali (*debunking*), seppur non sufficiente [4]. In entrambi i casi, l'efficacia degli interventi di *debunking* e del disimpegno costante dipende fortemente dal contesto di somministrazione. Gli individui sviluppano scetticismo e rigetto verso informazioni percepite come artificiali o controllate.

Pertanto, è fondamentale che l'esposizione avvenga nell'ambiente nativo della piattaforma. Questo assicura che l'utente interagisca in maniera naturale, permettendo ai ricercatori di misurare in modo affidabile l'impatto delle manipolazioni sperimentali introdotte. Preservare la validità ecologica rappresenta quindi un requisito funzionale primario. I ricercatori necessitano di strumenti tecnologici capaci di manipolare i

contenuti in tempo reale, preservando l'autenticità dell'esperienza online.

Il presente lavoro si posiziona proprio in questo scenario. L'obiettivo è sviluppare un'architettura software in grado di intervenire in modo silente ma controllato sugli algoritmi di ranking delle piattaforme. La capacità di iniettare, riordinare o filtrare i contenuti nei *feed* dei Social Media, offre ai ricercatori lo strumento necessario per condurre esperimenti comportamentali, lasciando intatta l'esperienza utente originaria.

1.2 Il Divario Tecnologico

A livello giuridico, gli algoritmi di ranking costituiscono una proprietà intellettuale delle piattaforme. Essi operano come delle vere e proprie "scatole nere" inaccessibili a ricercatori e revisori governativi, pur avendo un forte impatto pubblico [1, 5]. Per mitigare questi problemi informativi, l'Unione Europea ha introdotto nuove normative, come il *Digital Services Act* (DSA), che impongono mandati di trasparenza e l'obbligo di fornire accessi controllati a ricercatori accreditati. Tuttavia, la ricerca rimane comunque limitata a causa di lunghi processi burocratici e tempi di risposta delle Big Tech, che ostacolano lo studio di fenomeni emergenti ed improvvisi. Un esempio evidente è il recente caso delle elezioni in Romania, dove l'opacità dell'algoritmo di TikTok ha consentito l'amplificazione ingiustificata di alcuni messaggi politici, favorendo alcuni candidati ed alterando il processo democratico [1]. Altri esempi includono i sistemi di raccomandazione YouTube che, secondo vari studi, sembrano favorire l'adesione degli utenti verso contenuti estremisti [7].

Le aziende custodiscono questi algoritmi come veri e propri segreti industriali. Per proteggere questi segreti, le piattaforme limitano l'accesso alle API (Application Programming Interface) pubbliche, ostacolando la ricerca indipendente. Anche quando accessibili, le API forniscono dati parziali o decontestualizzati che limitano gli *audit* esterni sui Social Media, poiché non catturano la reale esperienza utente ed impediscono la misurazione delle interazioni in tempo reale [1]. Inoltre, la personalizzazione dei contenuti complica ulteriormente l'indagine. Siccome l'algoritmo genera un *feed* unico per ogni utente, sarebbe necessario studiare molteplici soggetti simultaneamente.

L'utilizzo di estensioni browser risolve parzialmente questo problema, poiché operano direttamente sul dispositivo dell'utente superando i limiti introdotti dalla chiusura delle API. Tuttavia, i recenti standard di sicurezza dei browser introducono nuovi vincoli architetturali. Ad esempio, Google Manifest V3 blocca l'intercettazione e la modifica delle chiamate di rete, vietando l'utilizzo di metodologie tradizionali, come l'applicazione di tecniche *Man-In-The-Middle* sui *payload* JSON per l'alterazione e lo studio dei *feed* algoritmici.

1.3 Dichiarazione del Problema

La ricerca scientifica necessita di nuovi strumenti per effettuare studi ed esperimenti in ambienti ecologici, in tempo reale, senza la cooperazione delle piattaforme e nel totale rispetto dei vincoli di sicurezza imposti. Alcuni approcci recenti si basano comunque sull'intercettazione delle API, introducendo soluzioni complesse ma vincolate a piattaforme specifiche (es. X o YouTube) [5, 6]. Di conseguenza, i ricercatori sviluppano architetture *ad hoc* per ogni singolo studio, trovandosi costretti a "reinventare la ruota" per ogni nuova piattaforma indagata. Questa frammentazione metodologica duplica gli sforzi di sviluppo e rende i risultati degli esperimenti difficilmente comparabili tra loro, limitando la riproducibilità della ricerca indipendente.

1.4 Novità dell'Approccio

La novità di questa tesi risiede nella struttura architeturale del framework. Piuttosto che implementare uno script specifico per ogni singolo studio, il nostro approccio si concentra sulla creazione di un'infrastruttura unica per tutti gli esperimenti. Questo paradigma separa completamente la logica sperimentale dai dettagli implementativi della piattaforma, fornendo un modello standardizzato per lo studio e la manipolazione degli algoritmi di ranking.

1.5 Contributo

Il presente lavoro introduce iBEARer (*in-Browser Experiments on Algorithmic Re-Ranking*), un framework progettato per intervenire sugli algoritmi social. iBEARer abilita l'esecuzione di esperimenti comportamentali *in-the-wild* volti a studiare l'impatto dell'esposizione a specifiche informazioni (es. interventi di *debunking*). Il sistema offre i seguenti contributi metodologici e architettureali:

- **Framework di Sviluppo Generalizzato:** iBEARer è pensato per essere un framework indipendente dalla piattaforma e flessibile per molteplici design di ricerca. La logica dell'esperimento è interamente descritta in un file di configurazione standardizzato. Questo approccio minimizza lo sforzo del ricercatore e consente di replicare gli studi senza modificare il codice sorgente. In questo modo si superano i limiti di standardizzazione e condivisione degli esperimenti, un aspetto risultato critico nello studio dello Stato dell'Arte (SOTA) [3].
- **Framework di Ricerca Full-Cycle:** iBEARer automatizza l'intero ciclo di vita dell'esperimento, dall'assegnazione dei gruppi sperimentali alla somministrazione di questionari pre e post-intervento, fino alla raccolta della telemetria, semplificando notevolmente il lavoro del ricercatore.
- **Implementazione Pratica su Reddit:** iBEARer fornisce un'implementazione pronta all'uso per eseguire operazioni di riordinamento, iniezione, modifica o cancellazione di contenuti su Reddit. A tale scopo, il framework è adattato

per operare anche su *Single Page Application* (SPA) soggette a *Lazy Loading*, aggirando le limitazioni delle API chiuse [1].

- **Audit Attivo:** iBEARer segna il passaggio da un audit osservativo [2] a un audit attivo. Gli interventi permettono di testare ipotesi sulla sensibilità degli utenti rispetto ai contenuti proposti ed effettuare operazioni di *disengagement controllato* da community problematiche [4], rappresentando un vero e proprio strumento per la "Lotta alla Disinformazione".
- **Ricerca Indipendente:** Il framework abbatte le barriere tecnologiche ed economiche, permettendo a qualsiasi ricercatore, anche in assenza di competenze informatiche, di eseguire esperimenti *in-the-wild* senza il bisogno di costose o complicate collaborazioni con le Big Tech.

1.6 Implicazioni

L'adozione di iBEARer trasforma la ricerca comportamentale sulle piattaforme social. Il rilascio genera quattro implicazioni principali per la comunità scientifica:

- **Riproducibilità:** La standardizzazione del design sperimentale tramite file JSON garantisce la condivisione esatta degli esperimenti. I ricercatori possono replicare gli studi su piattaforme diverse e confrontare i risultati. L'estensione dell'esperimento su più social media richiede uno sforzo minimo. Anziché implementare da capo un intero sistema, iBEARer necessita esclusivamente dello sviluppo di un nuovo *Platform Adapter*.
- **Validità Ecologica:** L'esecuzione degli esperimenti direttamente nell'interfaccia nativa della piattaforma preserva il contesto naturale di navigazione. Questo approccio produce dati comportamentali realistici e supera i limiti degli ambienti simulati in laboratorio.
- **Interventi Etici:** L'architettura facilita la ricerca e lo sviluppo di tecniche di manipolazione per fini etici, come le strategie di *debunking* o la censura della disinformazione.
- **Studio dell'Impatto sul Pubblico:** Il sistema permette di quantificare gli effetti delle piattaforme social sugli utenti. In questo modo, i ricercatori possono studiare come gli algoritmi di ranking influenzino l'opinione pubblica e la società.

Capitolo 2

Related Work

2.1 Algorithmic Auditing

L'*Audit Study* è una metodologia di ricerca sperimentale originariamente sviluppata per le scienze sociali ed economiche, con lo scopo di rilevare e misurare la discriminazione sul campo. La metodologia confronta profili fittizi, manipolando e controllando le variabili in esame, per isolarne l'effetto sulle decisioni finali. La successiva diffusione dei Social Media ha trasferito l'applicazione degli Audit Studies anche nel mondo digitale. In questo contesto i soggetti valutatori non sono più individui umani (es. reparto delle risorse umane che seleziona i soggetti da assumere) ma algoritmi automatizzati (che selezionano i contenuti da mostrare). L'applicazione degli *Audit Studies* a questi sistemi definisce l'*Algorithmic Auditing*, una metodologia volta a indagare il comportamento delle piattaforme digitali [7]. L'Auditing Algoritmico risponde ad esigenze di ricerca e favorisce l'adesione a requisiti normativi. Ad esempio, il *Digital Services Act* (DSA) europeo impone alle piattaforme digitali obblighi di trasparenza sulla moderazione dei contenuti, sulla disinformazione e sui sistemi di raccomandazione.

L'Auditing Algoritmico si classifica principalmente in 3 categorie:

- **Sockpuppet Auditing:** I ricercatori programmano agenti automatizzati (bot) con profili socio-demografici e cronologie di navigazione ben definiti [1]. Questi bot interagiscono con la piattaforma in modo controllato, permettendo ai ricercatori di misurare le variazioni nel feed/algoritmo a seconda del profilo del bot [7].
- **Scraping Auditing:** I ricercatori estraggono i dati proposti dalla piattaforma per analizzare il modo in cui le informazioni vengono presentate (es. analizzando i risultati di ricerca su TikTok o X) [1, 6, 2].
- **Crowdsourced Auditing:** Utenti reali installano estensioni del browser sui propri dispositivi. I ricercatori raccolgono i dati di navigazione per confrontare il comportamento della piattaforma su demografie reali [2].

Il framework iBEARer si posiziona a metà tra lo Scraping Auditing e il Crowdsourced Auditing. Il sistema infatti, estrae i risultati di ricerca proposti dalle piattaforme tramite un'estensione del browser installata sui dispositivi di utenti reali.

2.2 Approcci Esistenti: Stato dell'Arte (SOTA)

Mentre gran parte degli *Audit Studies* si limita esclusivamente all'osservazione passiva dei dati, la manipolazione in tempo reale dei contenuti social rappresenta ancora oggi una frontiera metodologica rara e tecnicamente complessa, ostacolata dalle difese architetturali delle piattaforme.

Ad esempio, per studiare l'influenza che gli algoritmi social hanno sul comportamento umano è necessario condurre A/B Test in ambienti ecologici (def. 3.3.1). Per implementare questi esperimenti comportamentali e superare le restrizioni delle API, una specifica branca dello *Scraping Auditing* e del *Crowdsourced Auditing* introduce la manipolazione in tempo reale dell'interfaccia utente [2, 5], utilizzando le estensioni web per intercettare e sovrascrivere il traffico di rete (es. XMLHttpRequest o payload JSON) contenente l'inventario dei post [5]. Questa tecnica prende il nome di *Man-In-The-Middle* ed avviene prima che la piattaforma utilizzi i dati per il caricamento (*rendering*) della pagina finale (*Document Object Model* - DOM).

La classificazione dei pacchetti JSON intercettati viene poi delegata a server di backend esterni che spesso utilizzano *Large Language Models* (LLMs - la tecnologia alla base delle attuali Intelligenze Artificiali) per analizzare ed assegnare un punteggio ai singoli post sulla base di metriche specifiche (es. il livello di animosità politica) [5, 6]. Il backend esegue poi gli interventi, come il riordinamento (*re-ranking*) o la rimozione dei post, in base ai punteggi ottenuti [5].

Infine, il server restituisce il JSON alterato al client, permettendo al browser di completare il rendering della pagina e mostrare il feed modificato all'utente.

2.3 Limiti degli Approcci Esistenti

Gli approcci metodologici discussi presentano tre limiti fondamentali:

- **Dipendenza dall'Intercettazione di Rete:** Le tecniche per l'Auditing indipendente si affidano prevalentemente all'intercettazione e alla modifica dei pacchetti di rete in transito prima del rendering grafico [5]. Tuttavia, l'introduzione di nuovi standard di sicurezza per le estensioni web, come Manifest V3, limitano queste tecniche per favorire la privacy degli utenti e migliorare i livelli di sicurezza [5].
- **Prestazioni:** L'impiego di *Large Language Models* per la classificazione e/o manipolazione in tempo reale, introduce severi problemi di latenza [5]. Il ritardo deriva dall'elevata complessità computazionale degli LLM che, sommata all'esecuzione asincrona delle piattaforme, aumentano in maniera evidente i tempi

di caricamento del *feed*, compromettendo l'esperienza utente e invalidando la natura ecologica dell'esperimento.

- **Accoppiamento Architetture:** Molte implementazioni attuali sono concepite per scopi di ricerca specifici e circoscritti, integrando la logica sperimentale e le direttive della singola piattaforma direttamente nel codice (*hardcoding*) per semplicità e convenienza. Questo approccio rende difficile la comprensione e la riproducibilità su piattaforme diverse [6, 2, 3, 7].

Capitolo 3

Formalizzazione e Definizioni

Questo capitolo definisce i concetti teorici, le astrazioni e le strutture dati alla base del framework iBEARer. La Sezione 3.1 descrive il ciclo di vita del partecipante, mentre la Sezione 3.2 delinea i requisiti funzionali, non funzionali e desiderati dell'infrastruttura. Infine, la Sezione 3.3 fornisce le definizioni formali e metodologiche del sistema.

3.1 User Journey

Il percorso del partecipante si articola in fasi sequenziali che garantiscono la continuità dell'esperimento:

- **Enrollment:** L'utente installa l'estensione. Al primo avvio, il backend assegna un identificativo univoco (UUID) e determina il gruppo sperimentale (es. controllo o trattamento), garantendo coerenza per l'intera durata dello studio.
- **Pre-Survey:** Prima di iniziare con l'esperimento, l'utente deve compilare un questionario per acquisire informazioni psicologiche e ideologiche (es. orientamento politico o propensione al complottismo) da usare successivamente nell'analisi dei risultati.
- **Osservazione e Intervento:** L'utente naviga normalmente sulla piattaforma. Il framework raccoglie la telemetria e, se scatta qualche trigger, applica gli interventi.
- **Post-Survey:** Al termine dell'esperimento, il sistema blocca l'interfaccia e richiede la compilazione di un questionario finale. Le risposte del questionario finale, combinate con la Telemetria e le risposte del questionario iniziale, consentiranno l'analisi finale dei risultati.
- **De-Enrollment:** Si rivelano le finalità dell'esperimento e si forniscono le istruzioni per la rimozione dell'estensione.

3.2 Requisiti del Framework

Per supportare il design sperimentale e consentire l'esecuzione di *Audit Studies* controllati, il framework iBEARer deve soddisfare una serie di requisiti formali.

- **Requisiti Funzionali:** Il framework deve supportare *Audit Studies* e *A/B Testing* sul campo, mirati ad analizzare l'impatto degli algoritmi e della disinformazione online. Per operare efficacemente all'interno di social media con feed dinamici, l'architettura deve preservare la validità ecologica della piattaforma, offrendo al partecipante un contesto di navigazione indistinguibile da quello quotidiano, così da evitare bias comportamentali o percezioni di controllo. Allo stesso tempo, il sistema deve abbattere le barriere d'accesso tecnologiche per permettere a ricercatori, scienziati sociali o esperti di dominio di configurare e condurre esperimenti senza la necessità di competenze software.

Sul piano del design sperimentale, il framework deve garantire la gestione dell'intero ciclo di vita del partecipante (arruolamento, valutazione psicometrica, manipolazione, osservazione e debriefing etico) offrendo al contempo la capacità di distinguere tra tempo assoluto e tempo effettivo di utilizzo della piattaforma.

Il sistema deve tracciare sia eventi di navigazione, come il caricamento di una pagina, sia eventi di interazione dell'utente, come i click. Inoltre, deve essere possibile ricollegare ogni azione ad uno specifico utente, permettendo di unire i dati della navigazione con le variabili psicologiche raccolte nei questionari. Tutto questo preservando l'anonimato e la privacy del soggetto.

Infine, per operare le manipolazioni, il framework deve fornire un set di interventi predefiniti per la modifica della pagina web, tra cui: inserimento, modifica, rimozione o riordinamento dei contenuti. Per preservare la validità ecologica è necessario che il sistema minimizzi qualsiasi anomalia visiva o latenza derivante dalle manipolazioni, garantendo che l'esperienza d'uso sia indistinguibile da quella nativa.

- **Requisiti Non Funzionali:** Il framework deve conformarsi ai moderni vincoli di sicurezza per le estensioni web. Per garantire la riproducibilità dell'esperimento, il framework deve disaccoppiare la logica sperimentale dall'implementazione grafica della piattaforma. Inoltre, deve garantire l'isolamento dei componenti per prevenire conflitti nell'ambiente di esecuzione. Inoltre, il sistema deve assicurare l'esistenza di una singola istanza di configurazione coerente e condivisa tra tutti i moduli.

In termini di prestazioni computazionali, il client deve minimizzare l'impatto sulla CPU durante la navigazione, gestendo i comportamenti asincroni delle piattaforme ed eseguendo gli interventi solo sui nuovi elementi caricati. Deve poi garantire l'efficienza della rete, per prevenire la saturazione o la perdita di dati. La sincronizzazione dei dati e i cambi di stato del partecipante devono essere comunicati in tempo reale. I canali di comunicazione devono essere protetti tramite appositi protocolli di autenticazione.

- **Desiderata:** L'architettura deve consentire l'estendibilità nativa a nuove piattaforme social tramite l'aggiunta di moduli di adattamento dedicati, senza richiedere modifiche al motore logico centrale. Infine, il framework deve garantire la totale indipendenza tematica dell'esperimento. Il ricercatore deve avere la possibilità di modificare l'argomento e il contesto dello studio aggiornando unicamente le parole chiave nel file di configurazione, senza la necessità di alterare o ricompilare il codice sorgente dell'applicazione.

3.3 Definizioni

3.3.1 Esperimenti Comportamentali Controllati ed A/B Test

Gli esperimenti comportamentali controllati sono metodologie di ricerca che isolano e manipolano specifiche variabili (X) per osservarne l'effetto sui comportamenti o sulle risposte (Y).

Un *A/B Test* è un esperimento controllato in cui gli utenti reali vengono divisi in due gruppi: controllo e trattamento. Al gruppo di controllo viene mostrata la versione attuale del sistema (A) mentre al gruppo di trattamento viene mostrata una versione differente (B), che è quella che si vuole testare. Lo scopo è determinare quale variante converte o performa meglio rispetto a un obiettivo specifico (es. quale produce più interazione o guida meglio l'utente verso il suo obiettivo). Il framework abilita l'esecuzione di questi studi attraverso il controllo rigoroso dei seguenti fattori sperimentali:

- **Variabile Indipendente:** è la variabile manipolata dal ricercatore per osservare un effetto sul sistema. Ad esempio, il modo in cui un contenuto social viene presentato.
- **Variabile Dipendente:** è la metrica misurata per quantificare l'effetto della Variabile Indipendente. Ad esempio, il numero di click su un contenuto social.
- **Covariata:** è una variabile esterna, non oggetto di manipolazione, che influenza la Variabile Dipendente. Il suo controllo statistico riduce la varianza d'errore e incrementa la precisione della stima dell'effetto causale. Ad esempio, il numero di contenuti caricati dalla piattaforma social durante l'esperimento.
- **Moderatore:** è una caratteristica preesistente del partecipante che si pensa alteri la relazione tra Variabile Indipendente e Variabile Dipendente. Ad esempio, l'orientamento politico del partecipante.

Mentre l'A/B testing valuta l'efficacia di una variante rispetto a conversioni immediate, l'esperimento controllato integra più gruppi di trattamento simultanei in periodi di osservazione longitudinali [6, 5, 1].

3.3.2 Formalizzazione dell'Esperimento

iBEARer definisce un Esperimento come un insieme di regole. Ciascuna Regola è una combinazione di tre elementi: Trigger, Condition e Intervention [3].

- **Trigger:** È un'azione specifica del browser o dell'utente. Ciascun Trigger viene rappresentato da un *Trigger Event*. Questo evento avvia il processo di valutazione delle condizioni. Se la valutazione restituisce esito positivo, il sistema applica gli interventi associati.
- **Condition:** È una singola condizione logica che può essere valutata solo vero o falso (*Boolean*). Il sistema valuta queste condizioni tramite operatori specifici (*Operators*) forniti dal framework. È anche possibile combinare più condizioni tramite operatori booleani (es. AND, OR).
- **Intervention:** È l'azione pratica da eseguire a seguito della validazione del trigger e delle condizioni.

Ad esempio, il caricamento della pagina dei risultati è un Trigger. La presenza di contenuti riguardanti Donald Trump è una Condizione. La rimozione di un contenuto è un Intervento. Combinando queste tre otteniamo la Regola:

"Se la pagina dei risultati (Trigger) presenta almeno un post riguardante Donald Trump (Condizione) allora rimuovi il post dalla lista dei risultati (Intervento)."

3.3.3 Formalizzazione di Telemetria ed Eventi

iBEARer registra le azioni dell'utente e i cambiamenti dell'interfaccia sotto forma di eventi. Un evento di telemetria rappresenta una singola variabile da tracciare. Il sistema distingue questi eventi in due categorie:

- **Eventi di Navigazione:** Tracciano i cambiamenti di stato della piattaforma. Non sempre dipendono dalla volontà dell'utente. Esempi di questa categoria includono il cambiamento dell'URL o il caricamento asincrono di nuovi contenuti nel feed.
- **Eventi di Interazione:** Tracciano le azioni esplicite svolte dall'utente. Esempi di questa categoria includono il click su un post, lo scroll della pagina o il tempo di permanenza su una specifica schermata.

Questa struttura permette di correlare ogni Intervento (Variabile Indipendente) con una specifica reazione comportamentale (Variabile Dipendente). Tale correlazione chiude il ciclo logico dell'esperimento e rende possibile la valutazione delle ipotesi.

Capitolo 4

Metodologia

Questo capitolo presenta la metodologia del lavoro in due parti. La prima parte (4.1) descrive l'architettura software di iBEARer: il modello client-server distribuito, il disaccoppiamento multiplatforma e la gestione dei flussi dati asincroni. La seconda parte (5.1) definisce il design sperimentale dello studio condotto su Reddit.

4.1 Architettura Software

iBEARer adotta un'architettura client-server distribuita in due parti:

- **Backend Django:** Gestisce la logica dell'esperimento in modo agnostico rispetto alla piattaforma. Il backend assolve quattro funzioni principali. Al primo avvio, registra il partecipante e ne determina il gruppo sperimentale. Dopo il completamento del questionario iniziale, filtra la configurazione del partecipante in base al gruppo sperimentale assegnato. Raccoglie la telemetria e notifica i cambi di stato del partecipante tramite *WebSockets*.

Il sistema utilizza Django REST Framework (DRF) e i relativi *Serializers* per controllare che i payload JSON in ingresso e in uscita rispettino gli schemi definiti nell'API e nel database. Questo meccanismo garantisce l'integrità dei dati scambiati tra client e server.

Per garantire l'esistenza di un'unica istanza del file di configurazione, condivisa e coerente tra tutti i moduli, il sistema applica il *Singleton Pattern* nel backend.

- **Estensione Chrome:** Costituisce il frontend ed esegue gli interventi sul dispositivo dell'utente. L'estensione opera mediante due contesti isolati: Il *Content Script* all'interno del DOM della pagina, per l'applicazione degli interventi grafici, ed il *Service Worker*, in un contesto separato, responsabile delle comunicazioni di rete. Tale separazione risponde a un vincolo di sicurezza del browser, che impedisce al *Content Script* di contattare direttamente il backend. La Sezione 4.3 ne descrive il meccanismo e i flussi di comunicazione.

L'estensione definisce poi un *Rules Engine*, che è il componente principale per il funzionamento dell'intero progetto. Questo componente interpreta a runtime le regole dichiarative definite nel *config.json* e valuta le triplette

Trigger-Condition-Intervention. Se la valutazione dà esito positivo, applica gli interventi associati.

Questa separazione concentra la logica sperimentale sul server e isola sul client i soli dettagli di manipolazione dell'interfaccia.

4.2 Configurazione dell'Esperimento

Il cuore del progetto risiede nel modo in cui si configura l'esperimento. L'infrastruttura separa la logica sperimentale dalle specifiche tecniche dell'estensione e dell'interfaccia ospite. A tale scopo, il sistema distingue tre livelli logici:

- **Platform Independent:** definisce i parametri generali dell'esperimento, indipendenti dalla piattaforma, come la durata complessiva, il numero di gruppi sperimentali e gli intervalli di sincronizzazione della telemetria. Questo livello è pensato per uso esclusivo del ricercatore. Separa il design sperimentale dall'infrastruttura tecnologica. Un sociologo può così definire o modificare le specifiche di un esperimento senza possedere alcuna competenza di programmazione e senza rischiare di compromettere la stabilità del software.
- **Platform Dependent Abstract:** definisce i trigger e gli interventi ad alto livello, mappandoli ai relativi moduli concreti con i rispettivi *Fully Qualified Names* (FQN). Rappresenta il punto di contatto tra l'astrazione dello studio e la sua implementazione tecnica.
- **Platform Dependent Concrete:** racchiude i selettori fisici del DOM (es. classi CSS e nodi HTML), confinati interamente nel *Platform Adapter* e nel codice degli Interventi. Questo livello è di competenza esclusiva dello sviluppatore software. I social media aggiornano costantemente le proprie interfacce grafiche, rischiando di rompere gli interventi basati sulla manipolazione DOM. A tale scopo, isolare i dettagli del DOM in questo livello permette di individuare subito le aree in cui operare: bisognerà aggiornare soltanto i selettori in questo livello, senza toccare il motore centrale o la logica dell'esperimento.

Questa separazione isola efficacemente i domini di competenza tecnica e sperimentale: uno sviluppatore può aggiornare i selettori o estendere il framework a nuovi social media senza conoscere i dettagli psicologici dello studio, mentre un ricercatore può progettare e lanciare complessi esperimenti comportamentali, senza dover scrivere una singola riga di codice.

4.2.1 Disaccoppiamento del Design Sperimentale

Il framework adotta un paradigma dichiarativo tramite un file di configurazione centralizzato che definisce esclusivamente le azioni da eseguire (Trigger, Condizioni, Interventi) omettendo la loro implementazione software [3]. Inoltre, la configurazione esclude qualsiasi riferimento a selettori o componenti specifici. I target sono identificati univocamente tramite una nomenclatura astratta, garantendo l'indipendenza dell'esperimento dalle continue modifiche strutturali dei Social Media [3].

4.2.2 Disaccoppiamento dalla Piattaforma Ospite

Il framework disaccoppia la logica sperimentale dall'interfaccia social tramite lo *Strategy Pattern* [3]. Il *Rules Engine* invoca esclusivamente i metodi esposti da una classe astratta generale, denominata *Platform Adapter*. Il sistema collega poi il livello astratto all'implementazione concreta mediante *Dependency Injection*. Il file `config.json` dichiara l'adapter specifico da caricare a runtime indicando il suo *Fully Qualified Name*.

Pertanto, il supporto a una nuova piattaforma richiede esclusivamente lo sviluppo di un nuovo *Platform Adapter*, senza alcuna modifica al motore di esecuzione o alla logica dello studio. Questo approccio preserva l'infrastruttura di rete, la gestione delle sessioni e i moduli di telemetria esistenti.

L'implementazione corrente fornisce un *Reddit Adapter* per operare sul feed dei contenuti della piattaforma.

4.3 Comunicazione e Flussi Dati

Questa sezione analizza l'infrastruttura di rete di iBEARer. Nei paragrafi successivi vengono dettagliati i protocolli per la comunicazione in tempo reale tra backend e frontend (4.3.1), le strategie adottate per superare i blocchi CORS (4.3.1), ed il modo in cui i componenti permettono la gestione del ciclo di vita del partecipante (4.3.3) e la relativa privacy (4.3.2).

4.3.1 Comunicazioni Client-Server

Il sistema sincronizza lo stato del partecipante in tempo reale tra backend e frontend tramite l'utilizzo di *WebSockets*.

Il backend adotta un'architettura asincrona basata sul framework Django Channels. Un modulo *ProtocolTypeRouter* smista il traffico di rete: inoltra le richieste HTTPS al server sincrono e reindirizza le connessioni *WebSocket* verso il server asincrono Daphne. La gestione delle connessioni persistenti e delle rotte è assegnata a moduli *Consumer* asincroni. Infine, l'infrastruttura implementa un *Channel Layer* per ricevere le comunicazioni asincrone da piattaforme esterne tramite WebHook (es. Google Forms) e notificare istantaneamente il client. Questa soluzione sostituisce i meccanismi tradizionali di *polling* HTTP, riducendo la latenza e ottimizzando il consumo di memoria sul server.

Il client, per evitare il sovraccarico della rete e prevenire la perdita di dati, accumula la telemetria in una coda locale, svuotata periodicamente solo se l'invio al server ha successo. Il backend ottimizza le scritture nel database eseguendo inserimenti cumulativi tramite la funzione `bulk_create` di Django.

Tuttavia, le policy di sicurezza del browser vincolano il flusso di rete dell'estensione. Il *Content Script*, responsabile delle manipolazioni DOM e dell'esecuzione degli interventi, viene eseguito direttamente nel contesto della piattaforma ospitante, ereditando tutte le caratteristiche, tra cui restrizioni di sicurezza. Di conseguenza, i vincoli di *Cross-Origin Resource Sharing* (CORS) bloccano qualsiasi richiesta HTTP diretta al nostro backend, impedendo il recupero della configurazione e l'invio della telemetria.

Per superare questa restrizione, il sistema delega la comunicazione di rete ad un *Service Worker*. Questa componente opera in un contesto isolato rispetto alla piattaforma ospite, disponendo di permessi propri e configurabili, che consentono di effettuare le chiamate HTTPS con il nostro server. Il *Content Script* e il *Service Worker* scambiano dati internamente tramite *Message Passing* del browser.

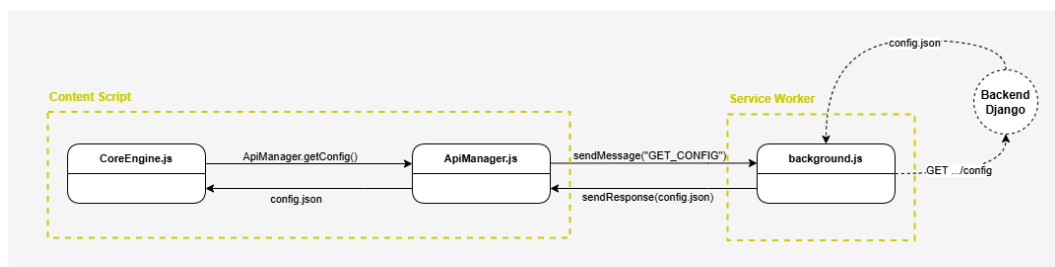


Figura 4.1. Flusso di recupero della configurazione tramite *Message Passing*. Il *Content Script* inoltra la richiesta al *Service Worker*, che esegue la chiamata HTTP verso il backend Django e restituisce la configurazione lungo il percorso inverso.

Infine, un sistema di autenticazione crittografica protegge gli endpoint di sincronizzazione da accessi esterni non autorizzati. Il Capitolo 6 approfondisce l'impatto di questa architettura nel superamento delle restrizioni di sicurezza di Manifest V3.

Al contrario, gli approcci documentati nello Stato dell'Arte [5, 6] non hanno incontrato questo problema poiché operano direttamente in pre-rendering, prima dell'applicazione dei vincoli di Manifest V3. Ricorrono al *Message Passing* esclusivamente per trasferire l'inventario dei post al server di backend esterno [5]. Questo approccio non è più praticabile con i moderni requisiti di sicurezza dei browser.

4.3.2 Gestione della Privacy dei Partecipanti

La conduzione di studi comportamentali sul campo impone il rispetto di stringenti vincoli etici e normativi. A tale scopo, il sistema non raccoglie Dati Personali Identificabili del partecipante, garantendo il completo anonimato.

- **Identificazione:** L'identità di ciascun partecipante è ricondotta esclusivamente a un identificativo numerico (*UUID*). Non è richiesta la raccolta di alcun attributo anagrafico.
- **Assegnazione del Gruppo:** L'*UUID* mappa in modo deterministico e permanente il partecipante a un gruppo sperimentale, assicurando coerenza per tutta la durata dello studio.

- **Integrazione dei Dati:** L'UUID permette di associare i risultati ottenuti dalle diverse fasi dello studio allo stesso partecipante. Ad esempio, consente di correlare i dati della telemetria con le risposte dei questionari.

4.3.3 Gestione del Ciclo di Vita dei Partecipanti

L'infrastruttura di rete appena descritta supporta in modo robusto l'intero ciclo di vita del partecipante:

- **Enrollment:** Come illustrato nella Figura 4.2, subito dopo l'installazione, il client richiede la configurazione iniziale al backend (1). Per preservare l'integrità dell'esperimento, il server fornisce un file `config.json` parziale, privo di trigger e interventi, poiché l'utente non possiede ancora un'assegnazione di gruppo (2). Conseguentemente, l'estensione richiede la creazione di un nuovo utente (3). Il backend dunque, genera l'UUID del partecipante e lo invia al client (4), che lo salverà in locale per includerlo nelle richieste future. Il sistema delega poi la compilazione del *Pre-Survey* a Google Forms (5). Al termine del questionario, Google Forms trasmette i dati al server tramite una richiesta HTTPS POST su un *WebHook* specifico (6). Il backend registra il completamento, assegna il gruppo sperimentale e notifica il client in tempo reale tramite *WebSocket* (7). Successivamente all'assegnazione del gruppo, il client effettua una nuova richiesta per ottenere la configurazione definitiva (8). A tale scopo, il backend filtra il `config.json` per includere solo le informazioni relative al gruppo del partecipante, e poi trasmette la configurazione definitiva (9).
- **Osservazione e Intervento:** Durante la navigazione, il *Rules Engine* elabora la configurazione localmente per manipolare il DOM, eliminando la necessità di interrogare il server per ogni singola azione. Parallelamente, i moduli Observer intercettano gli Eventi di Telemetria e li inseriscono in una coda locale. Dopodiché, il client invia la telemetria al backend in blocchi cumulativi tramite richieste HTTPS asincrone a intervalli regolari configurabili. Il server riceve i payload JSON, li valida e li inserisce nel database.
- **De-Enrollment:** Come illustrato nella Figura 4.2, quando il modulo *Timer-Manager* segnala lo scadere del timer sperimentale, l'estensione interrompe l'esecuzione di interventi, rimuove le manipolazioni e blocca il tracciamento di nuovi eventi. Il client svuota forzatamente la coda di telemetria per trasmettere gli ultimi eventi residui al backend (1) e reindirizza l'utente al *Post-Survey* su Google Forms (2). Analogamente alla fase iniziale, Google Forms notifica il backend al completamento del questionario tramite *WebHook* (3). Infine, il backend notifica il client del cambio di stato finale tramite *WebSocket* (4). A questo punto, il client esegue la pulizia completa della memoria locale.

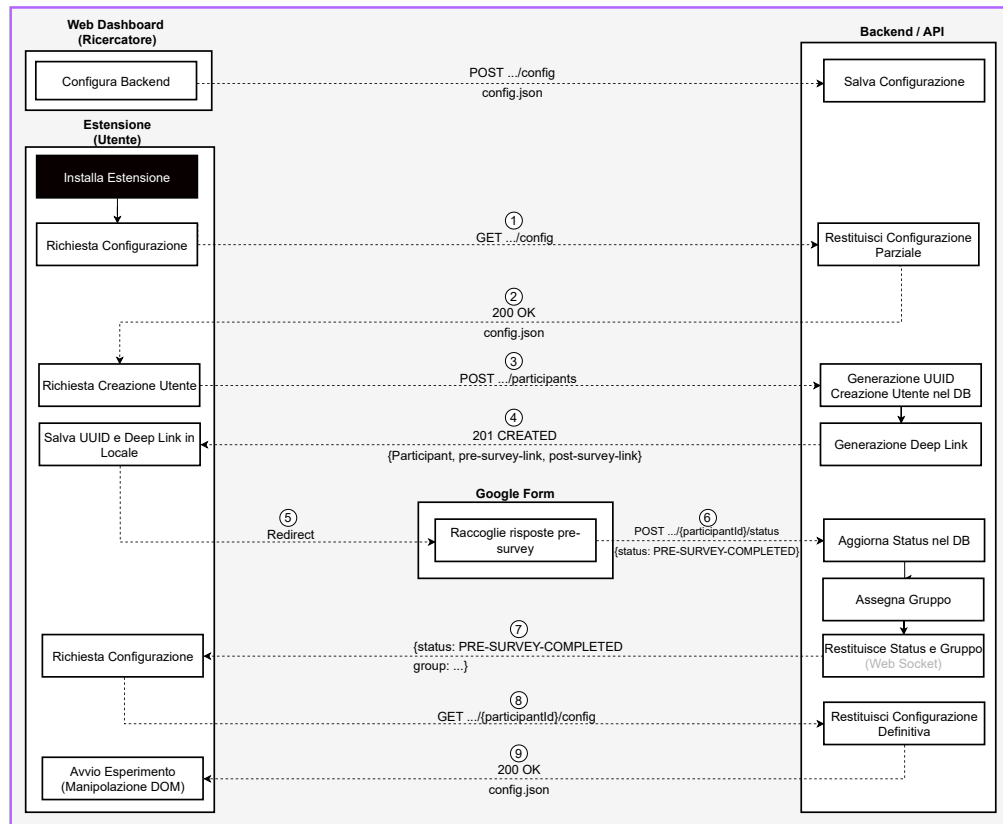


Figura 4.2. Diagramma di sequenza della fase di *enrollment*

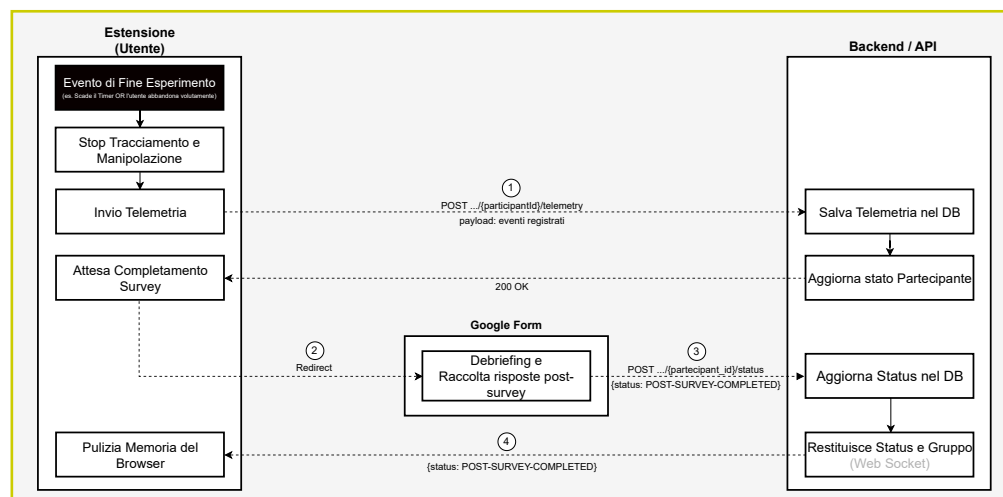


Figura 4.3. Diagramma di sequenza della fase di *de-enrollment*

4.4 Esempio Pratico di Applicazione

Per consolidare i concetti teorici e le definizioni di questo capitolo, presentiamo un esempio pratico di come iBEARer traduca l'astrazione architetturale in un esperimento reale. Ipotizziamo che un team di ricerca voglia studiare l'efficacia del *debunking* su Reddit, misurando se l'esposizione a fact-checking riduca l'interazione degli utenti con notizie complottiste.

4.4.1 Installazione ed Enrollment

L'utente volontario installa l'estensione iBEARer sul proprio browser (*Enrollment*). Al primo avvio, il sistema genera un UUID anonimo e lo assegna al partecipante in modo permanente. Prima di concedere l'accesso a Reddit, l'interfaccia vincola l'utente alla compilazione del *Pre-Survey* per valutare la sua propensione al complottismo (Moderatore). Dopo il questionario, il backend assegna all'utente il Gruppo Sperimentale. Se fa parte del gruppo di Trattamento allora visualizzerà gli interventi di debunking; Se fa parte del Gruppo di Controllo allora verranno solo registrati gli eventi di telemetria senza applicare nessun intervento.

4.4.2 Configurazione dell'Esperimento

Il ricercatore definisce l'intero studio in un file separato, senza modificare alcuna riga di codice:

- **Trigger:** Il framework rileva il caricamento asincrono di un nuovo post nel feed di Reddit.
- **Condition:** Il motore a regole valuta se il post proviene da una lista di subreddit monitorati (es. `r/conspiracy`) o se contiene specifiche parole chiave.
- **Intervention:** Se la condizione risulta vera, si inietta un banner informativo che smentisce le informazioni false, direttamente sotto il post. Questo intervento rappresenta la Variabile Indipendente dello studio.

4.4.3 Raccolta della Telemetria

Mentre l'utente naviga, il framework raccoglie la telemetria, tra cui: le query di ricerca, i risultati caricati dalla piattaforma, i click sui post, i click sugli interventi di debunking, il tempo passato su ogni pagina, i post visitati e le interazioni lasciate dall'utente (es. commenti o upvotes). Queste metriche comportamentali rappresentano la Variabile Dipendente dello studio.

4.4.4 Fine dell'Esperimento

Al termine del timer preimpostato, il framework blocca la navigazione sulla piattaforma e presenta il *Post-Survey* per misurare eventuali cambiamenti di opinione e rivelare le finalità dello studio. Guida poi l'utente nella rimozione dell'estensione.

Questo flusso dimostra come iBEARer riesca a gestire l'intero ciclo di vita di un Audit Study su piattaforme social.

4.5 Sfide Tecniche

Per aggirare i vincoli di sicurezza di Manifest V3, il sistema manipola esclusivamente il DOM post-rendering. Questa modalità espone l'architettura a diverse sfide tecniche:

- **Cambiamenti Strutturali:** le piattaforme social eseguono continui *A/B test* sulle interfacce, modificando classi CSS e tag HTML. Se non gestite, queste modifiche potrebbero rompere gli interventi interventi.
- **Caricamento Asincrono:** Per permettere l'*infinite scroll* dei feed, le piattaforme adottano meccanismi di *lazy loading* che caricano i contenuti in blocchi contigui man mano che si scorre la pagina. Questo significa che il sistema non può operare da subito sull'intero inventario dei post, poichè alcuni contenuti non sono ancora stati caricati.
- **Navigazione Single Page Application (SPA):** Le SPA aggiornano i contenuti dell'interfaccia sovrascrivendo porzioni del DOM senza mai ricaricare la pagina. Questo meccanismo impedisce il rilevamento automatico dei cambi di contesto, causando l'attivazione o la persistenza errata di moduli, che altrimenti sarebbero rimasti spenti.
- **Inquinamento del Global Scope:** In ambienti web complessi come quelli dei social media, iniettare script di terze parti rischia di causare collisioni nei nomi delle variabili, causando malformazioni o crash improvvisi dell'interfaccia.
- **Latenza di Rete:** L'integrazione di modelli di intelligenza artificiale per la generazione dinamica dei contenuti introduce una latenza di rete che, se non gestita opportunamente, può compromettere la navigazione e la validità ecologica dell'esperimento [5].

Le sezioni seguenti descrivono le implementazioni tecniche che hanno permesso di superare queste sfide.

4.6 Soluzioni Architetturali

iBEARer impiega soluzioni specifiche per operare in ambienti web ostili, caratterizzati da restrizioni di sicurezza e caricamenti asincroni. Questa sezione descrive le implementazioni pratiche che garantiscono il funzionamento e la validità degli esperimenti.

4.6.1 Robustezza degli Interventi sul DOM

Le piattaforme social applicano continui *A/B test* sui propri client, alterando la struttura del DOM. Pertanto, l'utilizzo di tag HTML o selettori CSS statici può causare la rottura della manipolazione anche solo passando da un partecipante all'altro. Per risolvere questo problema, iBEARer identifica i post tramite un algoritmo euristico, che non guarda ai selettori della pagina.

Ad esempio, per isolare il contenitore di un post, il sistema prende come riferimento un elemento stabile, presente in ogni post, cioè il titolo (`[data-testid="post-title"]`). Una volta trovato il nodo contenente il titolo, l'algoritmo risale ricorsivamente l'albero dei nodi. La risalita si arresta al primo nodo che contiene più di un titolo, identificato come il contenitore del feed (`mainFeedContainer`). L'algoritmo, quindi, restituisce l'ultimo nodo incontrato prima del `mainFeedContainer`. In questo modo siamo sicuri di estrarre il contenitore più grande che racchiude un singolo post, garantendo la stabilità e il funzionamento indipendentemente dal layout della pagina.

4.6.2 Gestione dell'Infinite Scroll

La maggior parte delle piattaforme social adotta meccanismi di *lazy loading* per inserire contenuti nel *feed* in modo incrementale, dando l'idea di *infinite scroll*. Allo stesso tempo però, l'estensione non può operare con i contenuti che non sono ancora stati caricati, riducendo di molto le opportunità dei ricercatori nelle manipolazioni sperimentali. iBEARer risolve questo problema con tre meccanismi complementari:

- **Pending Queue:** Gestisce l'iniezione o lo spostamento di un post verso una posizione che non è ancora stata renderizzata. In questi casi il sistema occulta temporaneamente il post di origine ed inserisce l'intervento nella Pending Queue. Quando l'utente scrolla, Reddit carica nuovi contenuti, attivando il `MutationObserver` che avvisa tutti gli altri moduli. A questo punto, il sistema verifica se la posizione target è stata renderizzata. In caso affermativo il sistema preleva l'azione dalla coda e la esegue; altrimenti, rimane in coda. Quando l'utente abbandona la pagina dei risultati (es. torna alla home page) la coda degli interventi viene svuotata per liberare memoria. Questo meccanismo permette di attendere lo scroll naturale dell'utente, senza alterare il comportamento della piattaforma.
- **Ghost Scroll:** Gestisce il caso inverso. Quando si vuole spostare un post dal basso verso l'alto è necessario che il contenuto target sia caricato subito. Pertanto, il sistema oscura momentaneamente la pagina dei risultati e simula uno scroll rapido verso il fondo, forzando la piattaforma a caricare i contenuti, fino al raggiungimento della posizione richiesta. Completato il caricamento, il sistema torna in cima alla pagina e mostra i contenuti manipolati. L'oscuramento dura pochi millisecondi e viene mascherato come ritardo di caricamento, preservando l'esperienza utente.
- **Cache del Browser:** Garantisce la coerenza delle posizioni casuali. Quando si esegue un intervento in cui le posizioni vengono assegnate in modo casuale, il sistema salva le posizioni assegnate nella memoria di sessione del browser, associandole alla specifica query di ricerca. Questo garantisce che, ad ogni rendering, refresh o scroll della pagina, il sistema riutilizzi le stesse identiche posizioni, evitando fenomeni di sfarfallio visivo.

4.6.3 Gestione delle SPA

Le Single Page Application (SPA) aggiornano i contenuti dell'interfaccia sovrascrivendo porzioni del DOM senza mai ricaricare la pagina. Di conseguenza, un Observer che dovrebbe essere attivo solo su una specifica pagina (es. solo nella pagina di ricerca) continua erroneamente a tracciare e manipolare i contenuti anche nelle pagine successive (es. la home page).

Per risolvere questo problema, iBEARer implementa un *SPA Watcher* che monitora i cambiamenti dell'URL e ripristina il contesto di Observer e Interventi ad ogni navigazione, azzerando anche i contatori di posizione. Questo componente mantiene la coerenza degli interventi durante l'intera sessione di navigazione.

4.6.4 Isolamento dell'Ambiente Globale

L'architettura isola i propri componenti dall'ambiente globale del browser mediante l'implementazione del *Registry Pattern*. Pertanto, il sistema memorizza eventi, observer, interventi e operatori in quattro sotto-registri isolati: **EventRegistry**, **ObserverRegistry**, **InterventionRegistry**, **OperatorRegistry**. Questa separazione previene qualsiasi conflitto di nomi con la piattaforma ospite.

Inoltre, ciascun modulo è identificato univocamente tramite il suo *Fully Qualified Name* (FQN). Per garantire la coerenza ed il corretto funzionamento tra il file di configurazione e l'estensione, ciascun modulo calcola il proprio FQN in modo automatico. Per fare ciò, il sistema genera volontariamente un errore e ne analizza lo **Stack Trace** per estrarre il percorso esatto del file che l'ha generata. Successivamente, l'estensione confronta questo percorso reale con la stringa dichiarata nel file di configurazione. Se i due non corrispondono, il sistema blocca l'esecuzione. Questa validazione impedisce l'attivazione di moduli inesistenti e garantisce la solidità dell'esperimento.

4.6.5 Iniezione Ibrida dei Contenuti

Il framework implementa un sistema di iniezione a due modalità, ottimizzato per bilanciare validità ecologica e ricchezza informativa:

- **Modalità Statica:** Gli attributi del post (titolo, autore, data, etc...) sono definiti direttamente nel file `config.json`. Se un attributo non venisse specificato il sistema utilizza dei valori di default. Questa modalità garantisce determinismo assoluto e latenza nulla.
- **Modalità Dinamica:** Il sistema estrae il contesto della pagina corrente e lo trasmette al backend. Il backend interroga un Large Language Model (default Gemini 2.5 Flash) forzando la restituzione di un oggetto JSON strutturato che verrà fuso con i dati statici ed iniettato sulla piattaforma. Quest'opzione sacrifica le prestazioni di rete a favore della coerenza contestuale. Per mitigare il problema della latenza, durante l'iniezione dinamica, il sistema applica una maschera grafica che simula un ritardo di caricamento.

La scelta tra iniezione statica o dinamica è lasciata al ricercatore nella fase di configurazione.

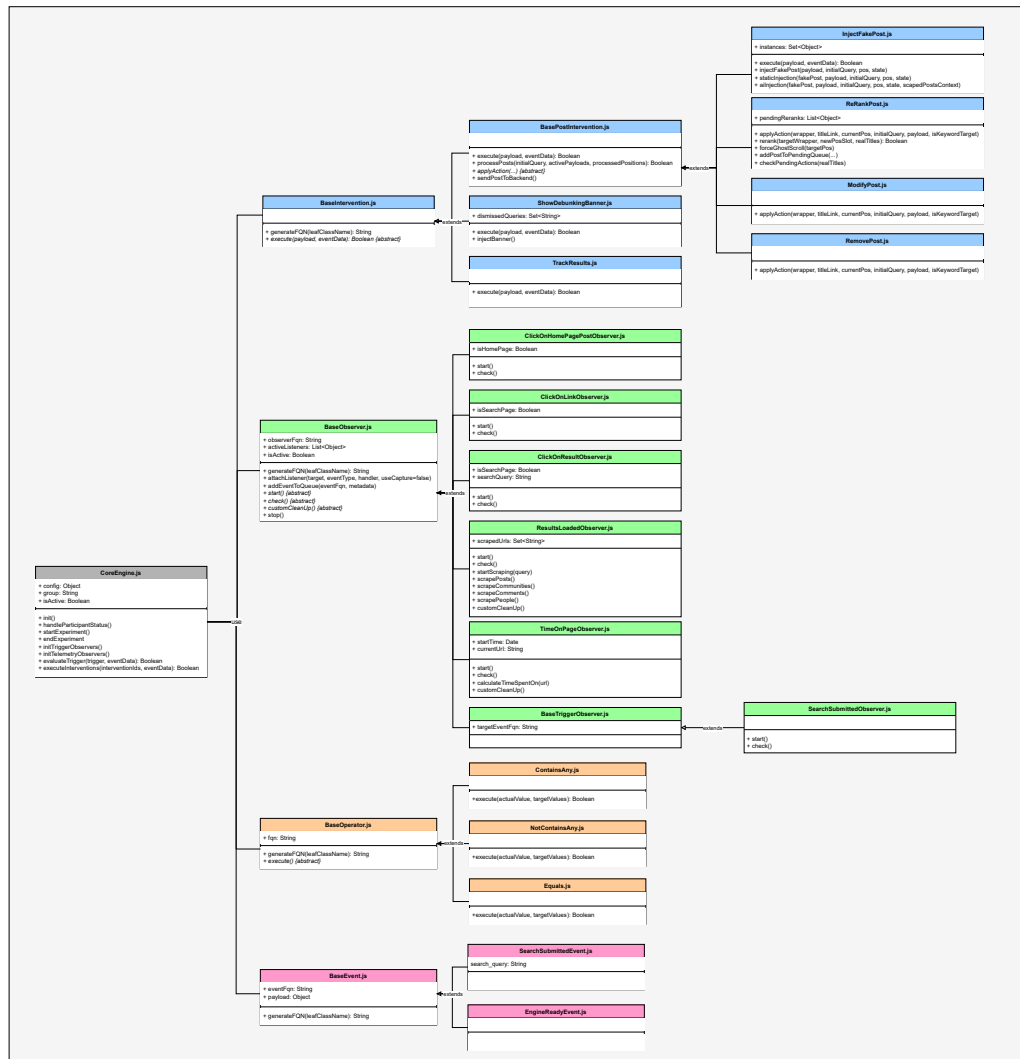


Figura 4.4. Diagramma delle Classi dell'estensione iBEARer.

4.7 Proprietà Architetture ed Estensibilità

L'obiettivo primario del progetto consiste nel fornire un'infrastruttura riutilizzabile per condurre *Audit Studies* su più piattaforme, superando il paradigma di script usa-e-getta. Per fare ciò si adotta l'architettura definita nel paragrafo 4.2 che disaccoppia l'implementazione concreta dalla configurazione astratta dell'esperimento. Tale architettura ha prodotto tre risultati ingegneristici principali:

4.7.1 Estensione ad un Nuovo Esperimento sulla Stessa Piattaforma

L'architettura separa completamente la logica dell'esperimento dall'implementazione del codice. Le specifiche dell'esperimento, le condizioni, i trigger e gli interventi sono tutti dichiarati in un file di configurazione esterno. Questo permette ai ricercatori di modificare il design sperimentale e distribuire gli aggiornamenti ai client, senza dover modificare il codice, ricompilare o reinstallare l'estensione. In questo caso, l'utente del framework non deve scrivere alcuna riga di codice sorgente, ma opera esclusivamente a livello dichiarativo, modificando il `config.json` per specificare i suoi Trigger-Condition-Intervention. Dopodichè, deve solo caricare il file di configurazione sul backend Django. All'avvio dell'estensione, il sistema convalida i *Fully Qualified Names* (FQN) e distribuisce automaticamente le regole ai client.

4.7.2 Aggiunta di Nuovi Interventi sulla Stessa Piattaforma

L'infrastruttura supporta l'integrazione di nuovi interventi di manipolazione senza alterare il codice principale. Per estendere le capacità del sistema, uno sviluppatore implementa una classe concreta che eredita dall'interfaccia `BaseIntervention.js`. Il nuovo modulo si registra autonomamente nell'`InterventionRegistry` attraverso il proprio FQN. Il *Rules Engine* rileva dinamicamente le nuove funzionalità e le esegue al verificarsi delle condizioni dichiarate nel file di configurazione, garantendo la scalabilità del codice sorgente.

4.7.3 Estensione dell'Esperimento su una Nuova Piattaforma

L'implementazione dello *Strategy Pattern* garantisce la completa separazione tra piattaforma ospite e funzionamento del codice. Il motore centrale gestisce il ciclo di vita dell'esperimento invocando metodi astratti del `PlatformAdapter`. È completamente agnostico rispetto alla piattaforma concreta. Pertanto, per estendere un esperimento verso una nuova piattaforma, è sufficiente implementare un nuovo Platform Adapter concreto ed aggiornare la relativa voce di dependency injection nel `config.json`. Questa struttura permette di estendere l'infrastruttura su nuovi social, mantenendo intatta l'infrastruttura di rete, di telemetria e il motore logico inalterati.

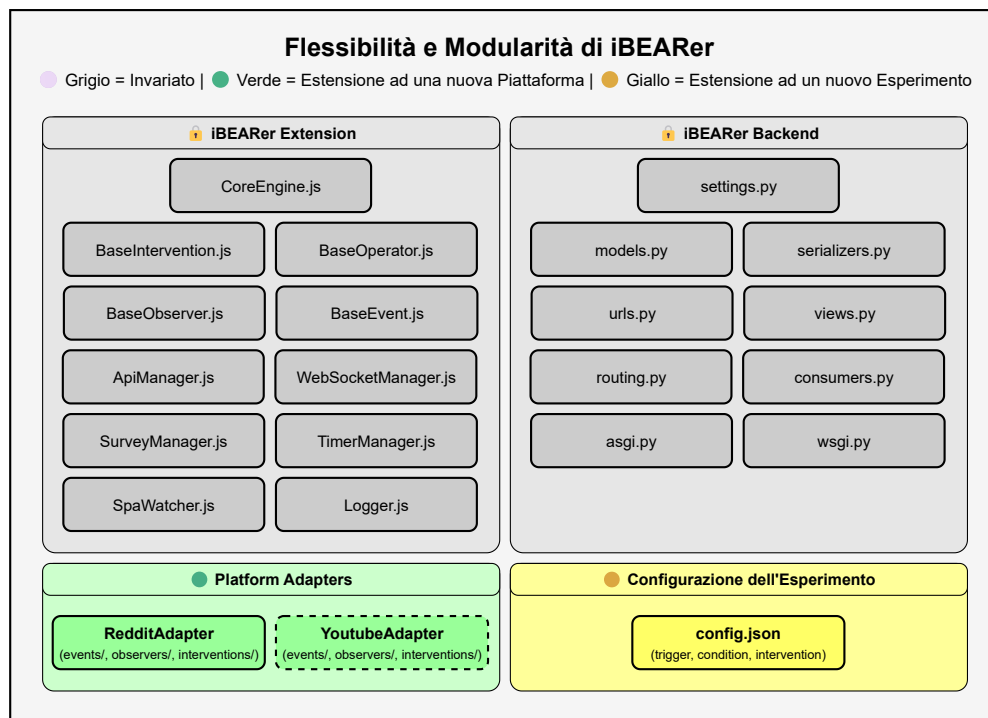


Figura 4.5. Organizzazione modulare dell'architettura iBEARer e impatto delle modifiche sul codice sorgente. In grigio sono rappresentati i componenti che rimangono invariati in ogni caso di estensione. In verde sono evidenziati i Platform Adapters, evidenziando come l'estensione a una nuova piattaforma, come Youtube, richieda la sola implementazione di un nuovo YoutubeAdapter. In giallo sono indicati i componenti da modificare per l'estensione a nuovi esperimenti. Si noti che è richiesta la modifica esclusivamente del file di configurazione.

Capitolo 5

Studio Empirico

Questo capitolo presenta un piccolo caso studio che dimostra le potenzialità, la flessibilità ed il funzionamento del framework. La prima sezione definisce il design sperimentale. Le sezioni successive descrivono il metodo di analisi e ne riportano i risultati.

5.1 Design Sperimentale

Sebbene iBEARer consenta di operare su molteplici piattaforme, in questo esperimento il sistema è configurato per operare solo su Reddit. Lo studio misura l'effetto che il *framing* dei contenuti ha sul comportamento degli utenti. Nel contesto della psicologia dei media, il *framing* di un contenuto rappresenta il modo in cui questo viene presentato o strutturato per influenzare la percezione o la reazione di chi legge.

5.1.1 Ipotesi di Ricerca

La letteratura scientifica sulle dinamiche dei social media evidenzia come gli algoritmi di raccomandazione tendano a premiare contenuti capaci di suscitare forti reazioni emotive [6, 1]. Partendo da questo presupposto, lo studio pone le seguenti ipotesi:

- **Ipotesi Diretta (HP1):** Si ipotizza che contenuti presentati con un *framing* sensazionalista o esagerato generino un livello di interazione significativamente maggiore rispetto allo stesso contenuto presentato in modo formale o giornalistico.
- **Ipotesi di Moderazione (HP2):** Poiché il contenuto target suggerisce il coinvolgimento di figure politiche, si ipotizza che i partecipanti con un basso livello di supporto verso tale figura presentino un tasso di interazione maggiore rispetto ai suoi sostenitori, mossi da un bias di conferma.
- **Ipotesi di Moderazione (HP3):** Si ipotizza che i partecipanti che ripongono un'alta fiducia nei social media, utilizzandoli come fonte primaria di informazione, siano maggiormente suscettibili al *framing* sensazionalistico.
- **Ipotesi di Moderazione (HP4):** Si ipotizza che i partecipanti caratterizzati da un'alta propensione alle teorie del complotto e un'alta chiusura mentale siano maggiormente suscettibili al *framing* sensazionalistico.

5.1.2 Configurazione dello Studio

Lo studio adotta un design *Within-Subjects*: ogni partecipante riceve entrambe le condizioni sperimentali nel corso della navigazione, permettendo di verificare come cambi il comportamento del singolo utente a seconda della presentazione del post.

5.1.3 Variabili e Moderatori

Lo studio controlla le seguenti variabili sperimentali:

- **Variabile Indipendente (X):** la tipologia di *framing* del post iniettato nel feed. Lo studio contrappone un titolo neutro, in stile giornalistico, a un titolo sensazionalista ed esagerato.
- **Variabile Dipendente (Y):** l'engagement dell'utente, misurato tramite il *Click-Through Rate* (CTR) sui post target.
- **Moderatori (Z):** i tratti psicologici preesistenti del partecipante, estratti dalla Pre-Survey tramite scale validate. Lo studio considera:
 - l'affinità politica verso Silvio Berlusconi, misurata tramite un *feeling thermometer*;
 - la propensione al complottismo (*Conspiracy Theory Belief*);
 - la chiusura mentale, misurata tramite la scala AOT-E (*Actively Open-minded Thinking about Evidence*) a punteggio invertito: un punteggio alto indica maggiore chiusura;
 - la fiducia nelle fonti di informazione, disaggregata in fiducia nei media tradizionali e fiducia nei social media.

5.2 Metodo di Analisi

Il sistema ha registrato 325 iniezioni. L'unione con le risposte ai questionari produce un dataset di 315 osservazioni. Ogni osservazione corrisponde a una singola *impression*: la presentazione di un post iniettato a un partecipante. La variabile dipendente è binaria: click (1) o assenza di click (0) sul post target.

L'analisi procede in tre fasi. Nella prima fase, un test χ^2 di indipendenza valuta l'associazione tra il tipo di *framing* e il click, verificando HP1. Nella seconda fase, una serie di modelli lineari generalizzati (GLM) con famiglia binomiale e funzione di collegamento logit, stima gli effetti dei moderatori, verificando HP2–HP4. Ciascun modello segue la specifica:

$$\text{click} \sim \text{tratto_z} \times \text{content_sensationalism}$$

dove `tratto_z` è il punteggio standardizzato del moderatore e `content_sensationalism` è un indicatore binario del tipo di *framing*.

Nella terza fase, si svolge un t-test post-hoc che confronta il CTR tra gruppi ad alto e basso punteggio per entrambi i framing di contenuto. Queste analisi esplorative verificano se il divario tra gruppi emerge selettivamente per una delle due condizioni.

5.3 Risultati

5.3.1 HP1: Nessun Effetto del Framing

Il test χ^2 non rileva alcuna associazione tra il tipo di framing e il click. Il CTR dei post giornalistici (42.9%) e quello dei post sensazionalisti (41.6%) non differiscono in modo significativo. HP1 non è supportata.

Questo risultato è confermato dai GLM: in tutti i modelli, il coefficiente del predittore `content_sensationalism` è prossimo a zero e non significativo (tutti $p > .60$). La tipologia di *framing*, nell'ambito di questo studio, non predice il click.

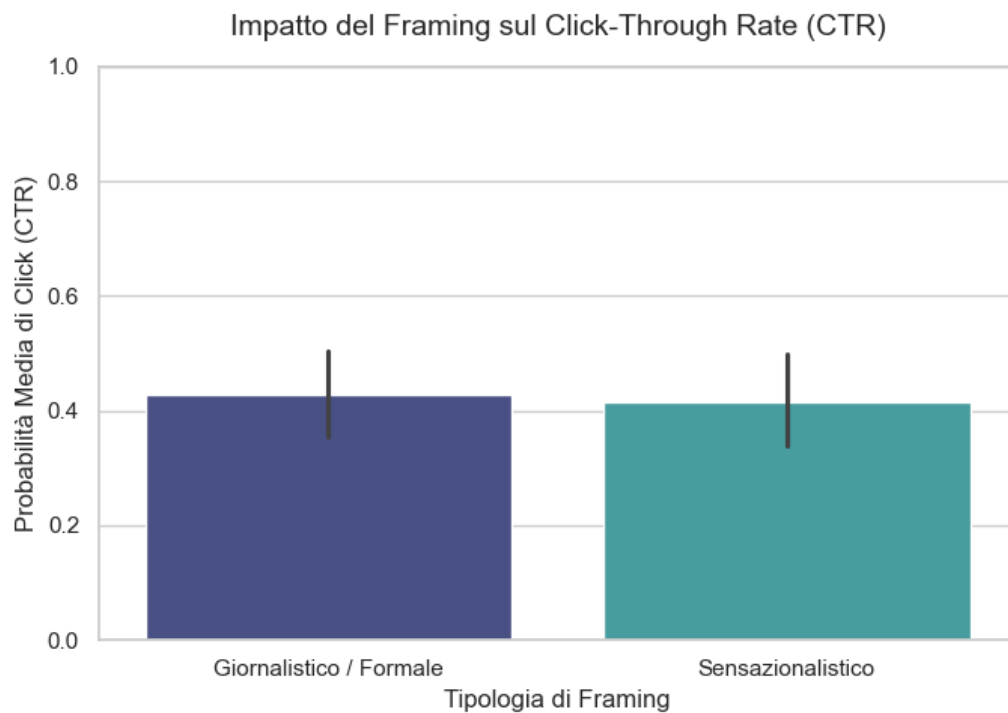


Figura 5.1. Impatto diretto del framing sul Click-Through Rate (CTR). Il grafico confronta la probabilità media di click tra la variante giornalistica e la variante sensazionalistica. Le barre verticali scure indicano gli intervalli di confidenza. Il grafico evidenzia un effetto diretto uniforme tra le due condizioni sperimentali, invalidando l'HP1.

5.3.2 HP2–HP4: Nessuna Moderazione

Nessuna delle interazioni tra moderatore e framing raggiunge la significatività statistica ($p < .05$). La Tabella 5.1 riporta i coefficienti dei termini di interazione per ciascun modello.

Tabella 5.1. Coefficienti dei termini di interazione (tratto $\times$ sensazionalismo).

Interazione	β	SE	z	p
Supporto Berlusconi $\times$ sensaz.	0.02	0.24	0.07	.948
Complottismo $\times$ sensaz.	0.24	0.23	1.04	.297
Chiusura mentale $\times$ sensaz.	0.40	0.23	1.73	.084
Fiducia media (totale) $\times$ sensaz.	-0.35	0.24	-1.46	.146
Fiducia media tradiz. $\times$ sensaz.	-0.16	0.26	-0.62	.535
Fiducia social media $\times$ sensaz.	-0.35	0.27	-1.31	.190

Se ne deduce che i tratti psicologici misurati non moderano l'effetto del framing sull'engagement. Pertanto, le HP2, HP3 e HP4 non sono supportate.

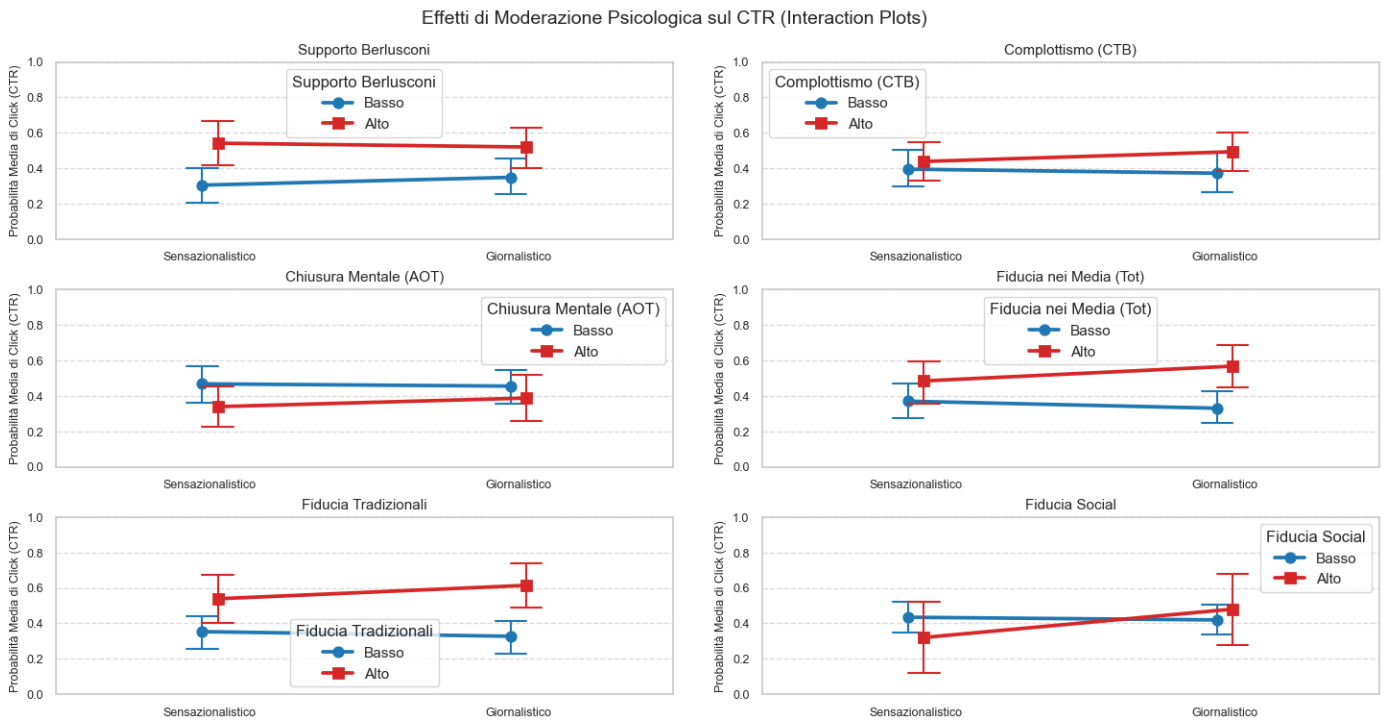


Figura 5.2. Effetti di moderazione psicologica e attitudinale sul Click-Through Rate (CTR).

I grafici di interazione scompongono la probabilità di click per livelli alti (linea rossa) e bassi (linea blu) dei tratti misurati nel pre-survey. I pannelli analizzano l'impatto del supporto politico, del complottismo (CTB), della chiusura mentale (AOT) e della fiducia nei media tradizionali e social.

5.3.3 Effetti Diretti dei Trattati sull'Engagement

Sebbene nessuna delle ipotesi venga supportata, si rilevano degli effetti diretti tra i tratti psicologici e la probabilità complessiva di click, indipendentemente dal tipo di contenuto. La Tabella 5.2 riporta gli effetti principali.

Tabella 5.2. Effetti principali dei moderatori.

Predittore	β	SE	z	p	IC 95%
Supporto Berlusconi	0.43	0.17	2.58	.010	[0.10, 0.75]
Chiusura mentale (AOT-E)	-0.36	0.17	-2.17	.030	[-0.69, -0.04]
Fiducia media tradizionali	0.43	0.18	2.38	.017	[0.08, 0.79]
Complottismo	-0.08	0.16	-0.47	.637	[-0.39, 0.24]
Fiducia media (totale)	0.29	0.17	1.75	.081	[-0.04, 0.62]
Fiducia social media	0.04	0.18	0.24	.813	[-0.32, 0.40]

Tre risultati emergono con chiarezza.

1. L'affinità politica verso Berlusconi è associata a un aumento significativo dell'engagement ($\beta = 0.43$, $p = .010$). Un alto livello di supporto produce una maggiore probabilità di click sui post che menzionano la figura politica, indipendentemente dal framing sensazionalistico o no. La direzione dell'effetto è opposta a quella ipotizzata in HP2: i sostenitori cliccano di più, non di meno.
2. La chiusura mentale è associata a una riduzione significativa dell'engagement ($\beta = -0.36$, $p = .030$). I partecipanti con un punteggio più alto sulla scala AOT-E, indicativo di una minore apertura mentale, mostrano una probabilità di click inferiore. Questo pattern è compatibile con un comportamento di evitamento: i partecipanti più chiusi tendono a non esplorare contenuti potenzialmente in contrasto con le loro convinzioni.
3. La fiducia complessiva nei Media non ci permette di estrarre effetti diretti, poiché non raggiunge la significatività ($p = .081$). Tuttavia, separando la fiducia nei media tradizionali dalla fiducia nei social media si osserva che la fiducia nei media tradizionali predice significativamente il click ($\beta = 0.43$, $p = .017$), mentre la fiducia nei social media non mostra alcuna associazione ($\beta = 0.04$, $p = .813$). I partecipanti che si fidano del giornalismo istituzionale esplorano i contenuti proposti con maggiore frequenza, mentre la fiducia nei social media non produce lo stesso effetto.

5.3.4 Analisi Post-Hoc: Sottoinsiemi per Tipo di Contenuto

Le analisi post-hoc esaminano se il divario tra gruppi ad alto e basso punteggio emerga selettivamente per i post sensazionalistici o per quelli giornalistici. La Tabella 5.3 riporta i risultati.

Tabella 5.3. Analisi post-hoc: CTR medio nei gruppi Alto e Basso per tipo di contenuto.

Moderatore	Contenuto	CTR Alto	CTR Basso	<i>t</i>	<i>p</i>
Supporto Berlusconi	Sensazionalistico	63%	25%	3.56	.002
	Giornalistico	38%	36%	0.10	.924
Fiducia media tradiz.	Sensazionalistico	54%	42%	0.81	.434
	Giornalistico	52%	37%	0.97	.352
Fiducia social media	Sensazionalistico	27%	57%	-1.96	.131
	Giornalistico	40%	40%	-0.03	.975

L'unico confronto significativo riguarda il supporto a Berlusconi sui post sensazionalistici ($p = .002$). I sostenitori raggiungono un CTR del 63%, contro il 25% dei non sostenitori. Sui post giornalistici lo stesso divario scompare (38% vs. 36%, $p = .924$). Il pattern suggerisce che l'affinità politica amplifica l'engagement in modo selettivo quando il contenuto è presentato con framing sensazionalistico. Tuttavia, questa osservazione va letta con cautela: il termine di interazione nel GLM corrispondente non è significativo ($p = .948$, Tabella 5.1). L'analisi post-hoc su gruppi dicotomizzati (Alto/Basso) può rivelare pattern che il modello su scala continua non cattura, ma la discrepanza impone di trattare questo risultato come esplorativo e descrittivo, non come evidenza di moderazione.

5.4 Sintesi

L'esperimento empirico produce diversi risultati.

1. **HP Diretta Non Supportata:** Il framing sensazionalista non incrementa l'engagement rispetto a quello giornalistico. Il CTR è quasi identico in entrambe le condizioni.
2. **HP di Manipolazione Non Supportate:** I tratti psicologici non moderano l'effetto del framing. Nessuna delle ipotesi tra moderatore e tipo di contenuto è significativa.
3. **Effetti Diretti:** L'affinità politica comporta un aumento del click. La chiusura mentale si associa a una riduzione del click, compatibile con un comportamento di evitamento protettivo verso contenuti potenzialmente dissonanti. La fiducia nei media tradizionali, a differenza della fiducia nei social media, comporta una maggiore esplorazione dei contenuti.

Tuttavia, nonostante i risultati non confermino nessuna delle ipotesi iniziali, lo studio dimostra chiaramente la capacità del framework di condurre esperimenti comportamentali completi in ambiente ecologico: dall'iniezione controllata di contenuti, alla raccolta della telemetria, fino all'analisi delle ipotesi.

Capitolo 6

Discussione

Questo capitolo interpreta i risultati architetturali e ne discute le implicazioni metodologiche. L'infrastruttura iBEARer dimostra la fattibilità di *Audit Studies* senza collaborazioni delle piattaforme esterne e nel pieno rispetto degli attuali vincoli di sicurezza. I paragrafi seguenti confrontano l'approccio proposto con lo stato dell'arte, evidenziando il superamento delle attuali limitazioni e le implicazioni che il sistema offre per la ricerca indipendente sui social media.

In questo modo, l'architettura di iBEARer dimostra che è ancora possibile condurre esperimenti ecologici senza dover dipendere dal rilascio di API ufficiali, spesso limitate o costose.

6.1 Manipolazione Post-Rendering

Lo stato dell'arte affronta la manipolazione dei feed prevalentemente tramite l'intercettazione lato client delle API di comunicazione (es. sovrascrivendo l'oggetto `XMLHttpRequest`) prima del rendering [5, 6]. L'introduzione di nuovi standard di sicurezza più restrittivi ed il crescente consolidamento delle difese delle piattaforme limitano l'efficacia di queste tecniche, spingendo la ricerca verso approcci alternativi [5, 1]. iBEARer risponde a questi vincoli con un approccio moderno e flessibile, basato sul disaccoppiamento modulare dei componenti e sull'esecuzione degli interventi in *post-rendering*, direttamente sul DOM lato client.

Tuttavia, operare sul DOM in post-rendering espone il framework alla volatilità strutturale delle interfacce web. Le piattaforme social modificano frequentemente il layout grafico, la gerarchia dei nodi HTML e i nomi delle classi CSS. Queste variazioni improvvise rischiano di rompere l'esecuzione.

L'architettura modulare di iBEARer mitiga questa vulnerabilità. Un'eventuale modifica strutturale del social media o dei requisiti di sicurezza richiederebbe esclusivamente la modifica dei parser grafici e dei selettori fisici all'interno del *Platform Adapter*, mentre il resto dell'infrastruttura (backend Django, Rules Engine, configurazione e telemetria) rimane completamente intatta.

Questo approccio garantisce la persistenza del framework a lungo termine e minimizza lo sforzo di manutenzione software.

6.2 Determinismo e Latenza

Il sistema iBEARer valuta le condizioni dei trigger tramite un *Rules Engine* deterministico che applica gli operatori logici dichiarati nel file *config.json*. Questo approccio diverge significativamente dai sistemi documentati nello stato dell'arte [5, 6]. Tali sistemi intercettano il traffico di rete in fase di pre-rendering e utilizzano *Large Language Models* (LLMs) per valutare e riordinare i contenuti in tempo reale.

Il determinismo di iBEARer riduce la flessibilità semantica della valutazione, ma produce valutazioni immediate che eliminano la latenza, mantenendo l'esperienza utente indistinguibile da quella nativa.

Il framework permette comunque la generazione di contenuti tramite intelligenza artificiale come modulo opzionale. In questo scenario il sistema applica delle maschere grafiche che simulano un ritardo di caricamento, con l'obiettivo di camuffare l'intervento. Quest'opzione sacrifica le prestazioni di rete a favore della coerenza contestuale.

6.3 Auditing Indipendente

Il Capitolo 1 illustra come la chiusura delle API pubbliche imponga ai ricercatori costose e vincolanti collaborazioni con le piattaforme, che innalzano barriere commerciali e logistiche per la ricerca indipendente.

Nonostante questo, il panorama dell'auditing algoritmico ha registrato un'evoluzione significativa con l'introduzione del *Digital Services Act* (DSA) nell'Unione Europea. Questo regolamento impone mandati di trasparenza che obbligano le piattaforme a fornire ai ricercatori accreditati l'accesso ai dati interni per monitorare rischi sistemici quali la polarizzazione e la disinformazione [2, 7].

Tuttavia, l'accesso ai dati garantito dal DSA Europeo è puramente osservativo, non consente di applicare manipolazioni sperimentali, limitando le possibilità dei ricercatori. Il processo rimane inoltre subordinato a lunghi processi burocratici e criteri aziendali che limitano l'indipendenza e la tempestività della ricerca, soprattutto se si considerano fenomeni social emergenti come la radicalizzazione politica e la diffusione di teorie complottistiche [2, 7].

Le piattaforme mantengono così il controllo segreto dei propri algoritmi raccomandazione. Tale situazione è aggravata ulteriormente se si considerano le logiche di monetizzazione dei social media, che spesso tendono a premiare i contenuti polarizzanti anche se problematici. Di fatto, gli algoritmi di raccomandazione, vengono strutturati per massimizzare il tempo di permanenza e le interazioni degli utenti (*engagement*), talvolta incentivando la diffusione di narrazioni complottiste o radicali in grado di generare reazioni emotive intense e, di conseguenza, un maggior numero di interazioni [7, 2].

L'architettura di iBEARer supera le limitazioni legate alla chiusura delle API e non richiede l'adesione ai lunghi processi burocratici imposti dal DSA. Operando direttamente sul DOM in fase di post-rendering, il sistema abilita un *audit* attivo e indipendente, senza mai interfacciarsi con il server remoto della piattaforma. Questa tecnica permette di rispettare i vincoli di sicurezza web, pur abbattendo le barriere economiche per la ricerca indipendente. Di fatto esonera i ricercatori dalla

necessità di accordi commerciali con le piattaforme, che spesso limitano il diritto alla pubblicazione scientifica.

Inoltre, il passaggio da un auditing di osservazione passiva ad un auditing di manipolazione attiva, consente applicazioni di rilevanza sociale. Tramite i suoi interventi dà ai ricercatori la possibilità di somministrare manipolazioni di debunking, disimpegno controllato da comunità problematiche, esposizioni a Exit Stories [4] e analisi della disinformazione [1].

6.4 Riproducibilità ed Estendibilità

La letteratura scientifica soffre di una profonda crisi di riproducibilità, aggravata dalla velocità con cui le piattaforme web si evolvono [3]. Gran parte degli strumenti documentati nello stato dell'arte integra la logica dell'esperimento direttamente all'interno del codice di manipolazione. Questa fusione tra regole sperimentali ed implementazione software produce due criticità principali: da un lato, rende difficile replicare fedelmente gli studi su campioni o periodi diversi; dall'altro, costringe a riprogettare l'intera infrastruttura software per ogni nuova piattaforma.

iBEARer risolve questo limite separando nettamente l'esperimento dall'infrastruttura. Da un lato, si inseriscono le specifiche dell'esperimento in un file di configurazione separato; dall'altro, si confina l'interazione con la piattaforma all'interno del codice dell'estensione. In questo modo il framework garantisce:

- **Riproducibilità:** Un esperimento si condivide e si replica trasmettendo solo il file di configurazione, senza modifiche al codice sorgente. Questo metodo abbassa notevolmente le barriere tecnologiche, incoraggiando la comunità scientifica non tecnica nella realizzazione di *Algorithmic Audit Studies*.
- **Estendibilità:** Spostare o estendere l'esperimento su una nuova piattaforma richiede esclusivamente l'implementazione di un nuovo *Platform Adapter*, mentre la logica sperimentale resta invariata.

6.4.1 Implicazioni per la Ricerca

L'infrastruttura sviluppata in questo progetto abilita nuove metodologie per l'indagine comportamentale in ambienti ecologici. La capacità di iniettare e riordinare contenuti dinamicamente permette di condurre esperimenti sistematici di *debunking* e di disimpegno controllato, superando i limiti dei metodi basati sulla sola osservazione passiva [4]. Inoltre, la gestione dell'intero ciclo di vita sperimentale (dall'enrollment, all'assegnazione dei gruppi, fino a questionari, telemetria, etc...) fornisce a futuri ricercatori l'infrastruttura per condurre studi su larga scala in modo replicabile, riducendo le barriere tecniche per l'auditing algoritmico.

Capitolo 7

Limiti e Sviluppi Futuri

Questo capitolo analizza i limiti architetturali e metodologici del framework derivanti dalla necessità di operare in modalità *post-rendering*. Tali vincoli espongono il sistema alle variazioni strutturali delle piattaforme ospiti. Vengono successivamente presentati gli sviluppi futuri orientati principalmente alla manutenibilità ed estendibilità.

7.1 Fragilità dell'Euristica DOM

Il sistema identifica i contenuti tramite un algoritmo euristico basato sulla struttura della pagina (es. ogni post ha il titolo) piuttosto che sui selettori CSS o sui nodi HTML. Questo approccio incrementa la robustezza degli interventi, poiché ignora i selettori CSS e i tag HTML che cambiano spesso (es. `<shreddit-post>`), ma introduce un nuovo limite funzionale. L'euristica vincola il funzionamento del sistema alla struttura della pagina. Una radicale riprogettazione dell'interfaccia richiederebbe un aggiornamento del codice.

Questo limite è conseguenza della manipolazione post-rendering, che rimane, però, l'unica via percorribile nel rispetto dello standard Manifest V3. La scelta è quindi ragionevole: il framework accetta il rischio di un aggiornamento futuro, in cambio della conformità alle politiche di sicurezza del browser.

7.1.1 Sviluppo Futuro: Riconoscimento tramite Computer Vision

Un possibile sviluppo futuro include l'utilizzo di modelli di *Computer Vision* per identificare i componenti visivi (es. post, commenti, reazioni) eliminando qualsiasi dipendenza dalla piattaforma, incluse le ancore strutturali.

7.2 Latenza di Rete nell'Iniezione Dinamica

La generazione dinamica dei contenuti tramite Intelligenza Artificiale richiede inevitabilmente tempi di esecuzione più lunghi, dovuti alla necessità di interfacciarsi con *Large Language Models* online. Per mitigare gli effetti di questa latenza, il framework applica maschere grafiche che simulano un ritardo di caricamento. Tuttavia,

nonostante il tentativo di camuffaggio, questa latenza compromette parzialmente la validità ecologica dell'esperimento, evidenziando una netta differenza prestazionale rispetto alla generazione statica.

7.2.1 Sviluppo Futuro: Esecuzione nel Browser

Per mitigare la latenza generativa dei modelli linguistici, uno sviluppo futuro può includere l'esecuzione dei modelli direttamente nel browser dell'utente tramite librerie ottimizzate in WebAssembly o ONNX Runtime. Questa configurazione elimina i tempi di trasferimento della rete, preservando la validità ecologica dell'esperimento.

7.3 Desincronizzazione dei Contenuti

La manipolazione DOM in *post-rendering* inserisce i contenuti solo graficamente all'interno dell'interfaccia. Pertanto, i filtri e gli algoritmi di ordinamento nativi della piattaforma ignorano tali elementi.

Questa condizione crea una desincronizzazione dei metadati. Le anteprime iniettate espongono valori statici (data, voti, commenti) definiti nella configurazione, che non variano nel tempo. Al contrario, le metriche dei post reali evolvono dinamicamente. Ciò genera un'incoerenza visiva tra l'anteprima iniettata ed il reale stato del contenuto sulla piattaforma. Queste limitazioni strutturali sono intrinseche all'approccio basato sul DOM. Come per la fragilità dell'euristica, il paradigma è il prezzo della conformità a i moderni standard di sicurezza

L'utilizzo di un'architettura *Man-In-The-Middle*, invece, evita questo problema: l'inserimento di un post nell'inventario avviene prima del *rendering*. Così facendo, la piattaforma elabora il post iniettato come fosse reale, applicando tutti i consueti processi di presentazione, inclusi ordinamenti e filtri.

7.3.1 Sviluppo Futuro: Forzare la Sincronizzazione

Per limitare l'incoerenza tra i metadati statici dei contenuti iniettati e le dinamiche reali della piattaforma, gli sviluppi futuri possono utilizzare due diverse strategie:

- **Aggiornamento dell'Anteprima:** Il modulo di iniezione aggiorna periodicamente i metadati (es. numero di voti e commenti) controllando i dati del post reale. In questo modo si può allineare l'anteprima iniettata con lo stato attuale del post, sacrificando le prestazioni.
- **Aggiornamento del Post Reale:** Il modulo di iniezione potrebbe estendere l'intervento anche alla pagina di destinazione. In questo modo, l'interfaccia visualizzata dopo il click utente risulterà coerente con l'anteprima manipolata, eliminando l'incongruenza.

7.4 Ambiente di Utilizzo

Poiché il sistema è stato sviluppato come estensione Chrome, il framework iBEARer opera esclusivamente su browser desktop basati su architettura Chromium. I sistemi operativi mobili impongono rigide restrizioni architetturali che non permettono l'installazione e l'utilizzo di estensioni web. Inoltre, meccanismi di sicurezza avanzati, come il *certificate pinning*, impediscono l'intercettazione e la manipolazione dei dati all'interno delle applicazioni ufficiali delle piattaforme [5]. Pertanto, l'infrastruttura iBEARer può operare esclusivamente su dispositivi desktop, introducendo potenziali bias demografici o comportamentali.

7.4.1 Sviluppo Futuro: Estensione ad Ambienti Mobile

Esistono principalmente due strategie per estendere il sistema agli ambienti mobile:

- **Adozione di Browser Mobile Dedicati:** Si potrebbe modificare il codice dell'estensione per operare in browser mobile che supportano le *WebExtensions*.
- **Sviluppo di applicazioni dedicate:** L'impiego di *WebView* permetterebbe di incapsulare la versione web della piattaforma ed iniettare la logica JavaScript direttamente lì [5]

Tuttavia, entrambe le soluzioni incrementano la complessità della fase di *enrollment*. La necessità di installare software non certificato o configurazioni di sicurezza non standard aumentano la diffidenza e l'impegno richiesto ai partecipanti, rendendo più difficoltoso il reclutamento dei partecipanti rispetto all'ambiente desktop.

7.5 Validazione Single-Platform

Nonostante iBEARer permette di estendere facilmente l'esperimento verso altre piattaforme, tramite la creazione di un nuovo Platform Adapter, l'implementazione presentata in questo documento fornisce solo un `RedditAdapter.js` e la configurazione dell'esperimento che misura l'impatto del framing sul Click-Through Rate (cap. 5). La validazione del framework rimane quindi circoscritta ad un singolo ecosistema social.

7.5.1 Sviluppo Futuro: Estensione a Nuove Piattaforme

Sviluppi futuri potrebbero consistere nella realizzazione di uno `YouTubeAdapter.js` per replicare l'esperimento sul *framing* dei contenuti su YouTube. Tale estensione consentirebbe anche di analizzare i sistemi di raccomandazione video, spesso associati a fenomeni di polarizzazione e radicalizzazione degli utenti [7, 2].

Capitolo 8

Conclusioni

La ricerca comportamentale sugli algoritmi di ranking necessita di strumenti per l'esecuzione di esperimenti in ambiente ecologico e in tempo reale. Tuttavia, la sempre più frequente chiusura delle API proprietarie e l'introduzione dello standard Manifest V3, bloccano l'intercettazione del traffico di rete, limitando fortemente gli studi sul campo. Inoltre, la gran parte delle soluzioni attuali opera su singole piattaforme o incorpora la logica dell'esperimento direttamente nel codice, limitando drasticamente la portabilità e la riproducibilità.

Questa tesi ha presentato iBEARer, un framework che supera i limiti introdotti da Manifest V3 operando manipolazioni dopo il rendering della pagina e direttamente sul DOM lato client. L'architettura utilizza un motore a regole deterministico che azzerla la latenza introdotta dai modelli di intelligenza artificiale per la valutazione dei contenuti, preservando la validità ecologica dell'esperienza utente. Inoltre, disaccoppia la logica sperimentale, definita in un file di configurazione separato, dalla logica specifica del social media, delegata a un *Platform Adapter*.

Sul piano tecnico, l'infrastruttura utilizza algoritmi euristici per l'individuazione di elementi nell'interfaccia, con lo scopo di aumentare la resistenza ai continui cambi di layout introdotti dalle piattaforme. Inoltre, il sistema gestisce il caricamento asincrono dei contenuti tramite l'utilizzo combinato di una *Pending Queue* e del *Ghost Scroll*. Infine, il framework isola i moduli dell'estensione dall'ambiente globale del browser tramite l'adozione del *Registry Pattern*.

Il principale obiettivo del framework è massimizzare l'estensibilità della ricerca. A tal proposito, i ricercatori possono modificare la logica sperimentale operando esclusivamente sul file di configurazione, senza intervenire sul codice sorgente. Allo stesso modo, lo sviluppo di nuovi interventi è indipendente dalla configurazione dell'esperimento e richiede l'implementazione di un nuovo modulo che estenda l'interfaccia base fornita. Infine, l'estensione a nuove piattaforme necessita del solo sviluppo di un *Platform Adapter* dedicato. Questo meccanismo abbate notevolmente le barriere tecnologiche per la ricerca sui social media, mentre l'applicazione degli interventi lato client rimuove la necessità di costose collaborazioni con le piattaforme proprietarie, abbattendo anche le barriere economiche.

In questo modo iBEARer trasforma l'auditing algoritmico da un'attività di osservazione passiva ad un'attività di intervento attivo. Le manipolazioni offerte permettono di studiare gli effetti sociali dei sistemi di raccomandazione ed apre le porte a numerose applicazioni, come esperimenti di *debunking* o disimpegno controllato.

Per validare l'efficacia del framework è stato impiegato in un esperimento su Reddit, con l'obiettivo di analizzare l'effetto che il *framing* di un post ha sull'*engagement* degli utenti e la sua moderazione da parte dei tratti psicologici del partecipante.

Gli sviluppi futuri, tra cui la realizzazione di un *Platform Adapter* per YouTube e l'estensione ad ambienti mobile, amplieranno ulteriormente le applicazioni di ricerca abilitate dal sistema.

Bibliografia

- [1] CHAN, J., CHOI, F., SAHA, K., AND CHANDRASEKHARAN, E. Examining algorithmic curation on social media: An empirical audit of reddit’s r/popular feed. *Proceedings of the International AAAI Conference on Web and Social Media*, **20** (2026), 391. doi:10.1609/icwsm.v20i1.42644.
- [2] GODINEZ, L. AND MUSTAFARAJ, E. Youtube and conspiracy theories: A longitudinal audit of information panels. In *Proceedings of the 35th ACM Conference on Hypertext and Social Media*, HT ’24, pp. 273–284. Association for Computing Machinery, New York, NY, USA (2024). ISBN 9798400705953. Available from: <https://doi.org/10.1145/3648188.3675128>, doi:10.1145/3648188.3675128.
- [3] HORTA RIBEIRO, M., WEST, R., LEWIS, R., AND KAIRAM, S. Post guidance for online communities. *Proc. ACM Hum.-Comput. Interact.*, **9** (2025). Available from: <https://doi.org/10.1145/3711046>, doi:10.1145/3711046.
- [4] PHADKE, S. Exit stories: Using reddit self-disclosures to understand disengagement from problematic communities. *Proc. ACM Hum.-Comput. Interact.*, **9** (2025). Available from: <https://doi.org/10.1145/3757592>, doi:10.1145/3757592.
- [5] PICCARDI, T., SAVESKI, M., JIA, C., HANCOCK, J. T., TSAI, J., AND BERNSTEIN, M. Reranking social media feeds: A practical guide for field experiments. *Trans. Soc. Comput.*, **9** (2026). Available from: <https://doi.org/10.1145/3800557>, doi:10.1145/3800557.
- [6] PICCARDI, T., SAVESKI, M., JIA, C., HANCOCK, J. T., TSAI, J. L., AND BERNSTEIN, M. S. Reranking partisan animosity in algorithmic social media feeds alters affective polarization. *Science*, **390** (2025), eadu5584. Available from: <https://www.science.org/doi/abs/10.1126/science.adu5584>, doi:10.1126/science.adu5584.
- [7] RIBEIRO, M. H., OTTONI, R., WEST, R., ALMEIDA, V. A. F., AND MEIRA, W. Auditing radicalization pathways on youtube. In *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency*, FAT* ’20, pp. 131–141. Association for Computing Machinery, New York, NY, USA (2020). ISBN 9781450369367. Available from: <https://doi.org/10.1145/3351095.3372879>, doi:10.1145/3351095.3372879.